Table of Contents

1. INTRODUCTION..... 4
    1.1. Post-Quantum Cryptography and Its Standardization..... 4
    1.2. General Overview of the ML-KEM Algorithm..... 5
    1.3. ML-KEM Security Levels..... 6
    1.4. Scope and Purpose of This Implementation..... 8
2. IMPLEMENTATION ARCHITECTURE..... 9
    2.1 Modules of this architecture..... 10
    2.2 Module ml_kem.c..... 12
    2.3 Module ml_kem_create_keys.c..... 13
        2.3.1. int ml_kem_create_ctx_struct(struct ml_kem_temp *temp, struct ml_kem_ctx *ctx, ml_kem_entropy_fn entropy)..... 15
        2.3.2. struct ml_kem_temp *ml_kem_temp_alloc(enum ml_kem_k level)..... 16
        2.3.3. struct ml_kem_ctx *ml_kem_ctx_alloc(enum ml_kem_k level)..... 16
        2.3.4. void ml_kem_destroy_temp_struct(struct ml_kem_temp *temp)..... 17
        2.3.5. void ml_kem_destroy_ctx_struct(struct ml_kem_ctx *ctx)..... 17
        2.3.6. int ml_kem_generation_entropy(struct ml_kem_temp *temp, ml_kem_entropy_fn entropy)..... 18
        2.3.7. void ml_kem_create_vector_poly(struct ml_kem_temp *temp)..... 18
        2.3.8. int ml_kem_create_public_key(struct ml_kem_temp *temp)..... 19
        2.3.9. void ml_kem_convert_to_bytes_format_and_copy(struct ml_kem_temp *temp, struct ml_kem_ctx *ctx)..... 20
        2.3.10.int ml_kem_finally_create_z_and_h_pk(struct ml_kem_ctx *ctx, struct ml_kem_temp *temp, ml_kem_entropy_fn entropy);..... 20
    2.4. ml_kem_encaps.c..... 21
        2.4.1. struct ml_kem_encaps_ctx *ml_kem_alloc_encaps(u8 *public_key_msg, enum ml_kem_k k)..... 22
        2.4.2. void ml_kem_wipe_encaps(struct ml_kem_encaps_ctx *ctx)..... 23
        2.4.3. void ml_kem_destroy_encaps(struct ml_kem_encaps_ctx *ctx)..... 23
        2.4.4. void ml_kem_encapsulation(struct ml_kem_encaps_ctx *ctx, u8 *msg, u8 *hash_pk, ml_kem_entropy_fn entropy)..... 23
        2.4.5. void ml_kem_unpack_pk_t_u16(struct ml_kem_encaps_ctx *ctx)..... 24
        2.4.6. void ml_kem_create_temp_secret_y(struct ml_kem_encaps_ctx *ctx, u8 *seed)..... 25
        2.4.7. int ml_kem_create_u(struct ml_kem_encaps_ctx *ctx, u8 *seed)..... 25
        2.4.8. void ml_kem_create_v(struct ml_kem_encaps_ctx *ctx, u8 *seed)..... 26
        2.4.9. void ml_kem_compress_ciphertext(struct ml_kem_encaps_ctx *ctx)..... 26
        2.4.10. void ml_kem_compress_and_pack(u8 *out, const u16 *coeffs, size_t n, u8 d)..... 26
        2.4.11. u16 ml_kem_compress(u16 x, u8 d)..... 27
        2.4.12. void ml_kem_create_u_full_matrix(struct ml_kem_encaps_ctx *ctx, u8 *seed);..... 27
    2.5. ml_kem_decrypt.c..... 27

2.5.1. void ml_kem_decrypt(struct ml_kem_decrypt_ctx *ctx_decry, u8 *ciphertext).. 28
2.5.2. void ml_kem_wipe_decrypt(struct ml_kem_decrypt_ctx *ctx_decry).....29
2.5.3 void ml_kem_decrypt_get_poly(struct ml_kem_decrypt_ctx *ctx_decry).....29
2.5.4 void ml_kem_decrypt_get_seed_m(struct ml_kem_decrypt_ctx *ctx_decry). 30
2.5.5. void ml_kem_decompress(struct ml_kem_decrypt_ctx *ctx_decry)..... 30
2.5.6. void ml_kem_decompress_one_poly(u16 *poly, u8 *compress_poly, const u8 d)..... 30
2.5.7. u16 ml_kem_decompress_one_coeff(u16 y, u8 d_u)..... 31
2.5.8. u8 ml_kem_decode1_bit(u16 w)..... 31
2.6. ml_kem_decaps_pool.c..... 31
2.6.1. struct ml_kem_general_keys_decaps_ctx *ml_kem_alloc_decaps_pool(enum ml_kem_k level, size_t ml_kem_pool_count, struct ml_kem_ctx *private_keys);.... 33
2.6.2. void ml_kem_pool_destroy(struct ml_kem_general_keys_decaps_ctx *head_pool)..... 34
2.6.3. struct ml_kem_general_keys_decaps_ctx *ml_kem_alloc_head_decaps_pool(size_t ml_kem_pool_count)..... 34
2.6.4. u16 *ml_kem_get_mem_two_bytes_ptrs_encaps(struct ml_kem_encaps_ctx *ctx, u16 *two_bytes_ptrs, enum ml_kem_k level).....35
2.6.5. u16 *ml_kem_get_mem_two_bytes_ptrs_decrypt(struct ml_kem_decrypt_ctx *ctx, u16 *two_bytes_ptrs, enum ml_kem_k level).....35
2.6.6. u16 *get_full_matrix(struct ml_kem_ctx *ctx_keys, struct ml_kem_general_keys_decaps_ctx *ctx_global);..... 36
2.7. ml_kem_ntt_main.c..... 36
2.7.1. Constant tables..... 37
2.7.2. void ml_kem_ntt_fips203(u16 f[ML_KEM_N]).....37
2.7.3. void ml_kem_intt_fips203(u16 f[ML_KEM_N]).....38
2.7.4. void ml_kem_multiply(u16 result[ML_KEM_N], u16 first[ML_KEM_N], u16.. 38
second[ML_KEM_N])..... 38
2.7.5. void ml_kem_add(u16 result[ML_KEM_N], u16 first[ML_KEM_N], u16..... 39
second[ML_KEM_N])..... 39
2.7.6. Interaction with other modules..... 39
2.8. ml_kem_shake.c.....40
2.9. ml_kem_sha3.c..... 41
2.10. keccak_1600.c.....42
2.11. ml_kem_encaps_pool.c.....43
2.11.1. ml_kem_create_all_structs().....43
2.11.2 ml_kem_assign_encaps_pool_u16_buffers().....44
2.11.3. ml_kem_assign_encaps_pool_u8_buffers().....44
2.12. Structure Description..... 45
2.12.1. ml_kem_ctx..... 45
2.12.2. ml_kem_temp.....47
2.12.3. ml_kem_encaps_ctx..... 49

2.12.4. ml_kem_decrypt_ctx..... 52
2.12.5. ml_kem_decaps_ctx..... 54
2.12.6. struct ml_kem_general_keys_decaps_ctx..... 56
2.12.7. struct ml_kem_encaps_slot..... 58
2.12.8. struct ml_kem_pool_encaps_ctx..... 59
2.13. Header ml_kem_core_header.h..... 60
2.13.1. List of constant..... 60
2.13.2. Inline functions in header..... 63
2.14. High-Level Execution Flow of ML-KEM in This Architecture..... 65
2.14.1. Encapsulation Flow..... 66
2.14.2. Encapsulation Flow with pool..... 67
2.14.3. Decapsulation Flow..... 68
2.14.4. Memory Reuse Strategy..... 71
2.14.5. Design Rationale..... 72
3. IMPLEMENTATION ARCHITECTURE DIAGRAMS..... 74

1. INTRODUCTION

This section provides a general introduction and an overview of ML-KEM, its purpose, and the motivation behind its development. If you are already familiar with these concepts (which is likely), you may skip this section. However, if you are interested in a deeper understanding of post-quantum cryptography and ML-KEM in particular, it is recommended to refer to more detailed literature on the subject. Relevant references are provided at the end of this documentation.

1.1. Post-Quantum Cryptography and Its Standardization

Modern public-key cryptography largely relies on mathematical problems that are considered computationally hard for classical computers. The most widely used cryptographic systems, such as RSA and elliptic curve cryptography (ECC), are based on the the difficulty of integer factorization and the discrete logarithm problem. However, the long-term security of these systems is threatened by the development of quantum computing. Quantum algorithms, most notably Shor's algorithm, demonstrate that both factorization and discrete logarithm problems can be solved in polynomial time on sufficiently powerful quantum computers.

In response to this threat, the field of Post-Quantum Cryptography (PQC) has emerged. Its goal is to develop cryptographic algorithms that remain secure against both classical and quantum adversaries. Unlike traditional systems, PQC algorithms rely on different mathematical assumptions that are currently believed to be resistant to known quantum attacks.

One of the key hardness assumptions used in PQC is the Module Learning With

Errors (Module-LWE) problem. The ML-KEM algorithm (Module-Lattice-based Key Encapsulation Mechanism) is based on this problem. ML-KEM is a standardized key encapsulation mechanism (KEM) that belongs to the class of post-quantum cryptographic algorithms. Its primary goal is to enable secure key exchange over an insecure communication channel. In modern cryptographic infrastructures, ML-KEM is intended to replace existing key establishment mechanisms based on:

- integer factorization (RSA)
- discrete logarithms (Diffie–Hellman, ECDH)

as these are vulnerable to quantum attacks.

To prepare cryptographic infrastructure for the potential emergence of practical quantum computers, the National Institute of Standards and Technology (NIST) initiated a standardization process for post-quantum cryptographic algorithms. As a result of this process, several algorithms were selected for future use. One of them is ML-KEM, standardized in FIPS 203, which defines:

- the algorithm specification
- security parameters
- encapsulation and decapsulation procedures

1.2. General Overview of the ML-KEM Algorithm

ML-KEM (Module-Lattice-based Key Encapsulation Mechanism) is a post-quantum cryptographic algorithm designed to establish a shared secret key between parties communicating over an insecure channel. The algorithm belongs to the class of Key Encapsulation Mechanisms (KEMs). The core idea of such mechanisms is that one party

generates a random symmetric key and securely transmits it to another party by encapsulating it using the recipient's public key. As a result, both parties derive the same shared secret, which can subsequently be used in symmetric cryptographic algorithms.

ML-KEM is based on the Module Learning With Errors (Module-LWE) problem, which is a structured variant of the Learning With Errors problem. This problem belongs to the class of problems related to lattice-based cryptography and is considered computationally difficult for both classical and quantum algorithms. This property makes it a suitable basis for constructing post-quantum cryptographic schemes.

The ML-KEM algorithm is a standardized version of the cryptographic construction CRYSTALS-Kyber, which was selected as the primary key encapsulation mechanism during the post-quantum cryptography standardization process. The construction relies on arithmetic operations over polynomials in a modular ring and makes extensive use of performance optimizations, most notably the Number Theoretic Transform (NTT), which significantly improves computational efficiency.

In general, the ML-KEM workflow consists of three main procedures:

- Key Generation, which produces a public and a secret key pair
- Encapsulation, where the sender uses the recipient's public key to generate a ciphertext and a corresponding shared secret
- Decapsulation, where the recipient uses the secret key to recover the same shared secret from the received ciphertext

1.3. ML-KEM Security Levels

The ML-KEM standard defines three parameter sets corresponding to different levels of cryptographic security and intended for use in different scenarios. These variants

are denoted as **ML-KEM-512**, **ML-KEM-768**, and **ML-KEM-1024**. All variants are based on the same cryptographic construction but differ in parameter values that affect the security level, key sizes, and computational complexity of the algorithm.

1. **ML-KEM-512** provides a baseline level of cryptographic security and is intended for systems where it is important to maintain a balance between performance and the size of cryptographic data. This level uses the smallest key and ciphertext sizes among all ML-KEM parameter sets, making it the most efficient in terms of computational cost.
2. **ML-KEM-768** provides a higher level of cryptographic strength and is considered a general-purpose compromise between security and efficiency. In many use cases, this level is regarded as the recommended choice for practical deployment.
3. **ML-KEM-1024** offers the highest level of security, using larger parameter values and, consequently, larger key and ciphertext sizes. This increases resistance to potential cryptanalytic attacks but also results in higher computational cost.

Thus, all three ML-KEM levels implement the same cryptographic construction while allowing the selection of parameters based on system requirements in terms of security, performance, and data size. It should be noted that the goal of this implementation is to provide a unified ML-KEM design that accepts the security level as a parameter. Based on the selected level, all computations are performed accordingly, rather than maintaining three separate implementations of the same algorithm for each security level.

1.4. Scope and Purpose of This Implementation

This work presents an implementation of the ML-KEM algorithm in two variants: a user-space version and a Linux kernel module. Both implementations are designed with a strong focus on portability across different environments and ease of further integration and porting. To achieve this, the code is written in C in accordance with the C11 standard, and the architecture is designed with consideration for low-level environments, particularly the specifics of the Linux kernel.

The implementation provides all core procedures of the ML-KEM algorithm, including key generation, encapsulation, and decapsulation of the shared secret, in accordance with the FIPS 203 specification. The coexistence of two implementation variants—user space and kernel space—introduces additional requirements for memory management, execution efficiency, and secure handling of cryptographic data. The overall architecture and internal structure of both implementations are described in the following sections of this documentation.

2. IMPLEMENTATION ARCHITECTURE

This is the third part of the documentation, it describes the implementation architecture. For better understanding and readability, it is recommended to open the diagrams from section 4. The structure of this document is as follows:

1. All modules are described. Modules are the parts into which this ML-KEM implementation is divided, each module performs a specific function (part of the logic) of the ML-KEM algorithm. From the point of view of the C language in which this implementation is written, it is simply a file with code that is part of a large project and connected into a single library.
2. All structures that combine logically similar data from the point of view of this algorithm and its current implementation are described. It is also indicated how modules use these structures.
3. The interaction and interdependence of structures are described.
4. The interaction and interdependence of modules (or the main functions of modules) are described.

It should be noted that a separate subsection is allocated for each module and for each structure. If during the reading you need to clarify another module or structure (for example, while reading the section on the key generation module, you want to learn in detail about the permanent or temporary structure, or for example, while reading about the API-level, which functions are called and how they work from the key generation module), then refer to the corresponding subsection. Also, if you want to go beyond the scope of this implementation and learn more about the logic of the algorithm and the mathematics on which it is based, then you should refer to the official ML-KEM standard.

2.1 Modules of this architecture

It should be noted that modules are a set of code files that implement a certain set of functions. The division into modules is due to considerations of logical organization and operation of the ML-KEM algorithm, how and why it was divided into modules in this way will be described in this subsection below.

This implementation of ML-KEM involves the breakdown into the following modules:

1. ml_kem.c is the main wrapper module that should be used by anyone who wants to use this implementation.
2. ml_kem.h is a header file with declarations of prototype functions of the main wrappers, which are implemented in the ml_kem.c module
3. ml_kem_create_keys.c — is a key generation module. This file implements a set of functions that, according to the standard, generate a private key vector s, and a public key pk, and fills the structure for storing keys and additional data in its fields — ml_kem_ctx. This module also uses the ml_kem_temp structure — this structure is created when generating keys and is destroyed when the generator is finished — this is a temporary structure that is needed to store temporary data that is needed only during the key generation process — therefore, temporary objects that are not needed after generation, which are zeroed and destroyed after the key generation operation is completed, are stored in this structure.
4. ml_kem_encaps.c is an encapsulation module, this module has an implementation of a set of functions that encapsulate data, for this purpose the ml_kem_encaps_ctx structure is created and used
5. ml_kem_decrypt.c is a decryption module, the set of functions of this module implements the decryption process (not to be confused with decapsulation) and does this using the ml_kem_decrypt_ctx structure

6. ml_kem_decaps_pool.c is a module that creates a pool, a set of slots, from combinations of encapsulation and decryption structures (ml_kem_encaps_ctx and ml_kem_decrypt_ctx), which it combines into a single structure for full decapsulation for each slot, thus - you can specify the number of slots in the pool relative to one key, that is, all these slots work with one ml_kem_ctx structure. This allows for parallel execution with a low number of allocations. How this is done will be described in the overview of this module. It operates with two structures - ml_kem_decaps_ctx (this structure is needed for each slot), and ml_kem_pool_decaps_ctx (the main pool structure and manages all slots).
7. ml_kem_ntt_main.c is a module that performs the functions of NTT transformation (butterfly algorithm), it is called by the ml_kem_decrypt.c, ml_kem_encaps.c and ml_kem_create_keys.c modules when calculating keys, ciphertext and shared secret. It is needed to optimize polynomial multiplication operations in the point domain - this is the idea of the algorithm itself and its optimization and acceleration.
8. ml_kem_shake.c is a module that implements SHAKE128 and SHAKE256, this is necessary to obtain from the seeds entire large sets of raw byte arrays, from which polynomials are then formed.
9. ml_kem_sha3.c is a module that implements the SHA3-256 and SHA3-512 algorithms — they are needed when generating seeds and hashes from the primary seed in key generation and encapsulation.
10. keccak_1600.c is a module for implementing the keccak algorithm, it is necessary for the ml_kem_sha3.c and ml_kem_shake.c modules, since without it it is impossible to perform these operations

11. ml_kem_encaps_pool.c It is an encapsulation pool module. It creates a pool of pre-prepared encapsulation structures, allocating memory and setting up structures

in advance to facilitate multi-threaded encapsulation.

12. ml_kem_core_header.h is a header file with declarations that declares all extern prototype functions, structures, and constants for all internal modules

This description is generalized, a more detailed description for each module is provided below in each subsection of this section, where all the functions operated by the module are also described.

2.2 Module ml_kem.c

This module represents the first (API) level of the implementation. The functions declared in ml_kem.h are intended to be used by users of this implementation. First, a description of all the functions of this module will be given, and then, in this subsection, their interaction will be described. The ml_kem.c module has the following functions:

1. struct ml_kem_general_keys_decaps_ctx *ml_kem_create_object(enum ml_kem_k level, size_t ml_kem_pool_count, ml_kem_entropy_fn entropy);
2. structml_kem_general_keys_decaps_ctx*ml_kem_create_object_from_imported_s k(u8 *sk, size_t size_sk, size_t size_pool);
3. void ml_kem_destroy_core(struct ml_kem_pool_decaps_ctx *ctx_pool);
4. int ml_kem_encaps_core(struct ml_kem_pool_encaps_ctx *ctx_pool, u8 *pk, size_t size_pk, u8 *buffer_ct, size_t size_ct, u8 *buffer_ss, size_t size_ss, ml_kem_entropy_fn entropy);
5. int ml_kem_decaps_core(struct ml_kem_pool_decaps_ctx *pool, u8 *ciphertext, size_t len_ciphertext, u8 *result, size_t len_result);
6. int ml_kem_get_public_key(struct ml_kem_general_keys_decaps_ctx *ctx, u8 *buffer_for_public_key, size_t size_buffer);

7. int ml_kem_get_sk(struct ml_kem_general_keys_decaps_ctx *ctx, u8 *buffer_for_sk, size_t size_buffer_for_sk);
8. struct ml_kem_pool_encaps_ctx *create_encaps_pool(size_t number_slots_512, size_t number_slots_768, size_t number_slots_1024);
9. void ml_kem_encaps_pool_destroy(struct ml_kem_pool_encaps_ctx *ctx);
10. void ml_kem_decaps_ct_select_ss(struct ml_kem_general_keys_decaps_ctx *ctx, u8 *ciphertext, size_t len_ciphertext, u8 *result, size_t curr_slot);
11. size_t get_s_vector(const u16 *secret, u8 *buffer, enum ml_kem_k k);
12. size_t unpack_s_vector(u16 *secret, u8 *buffer, enum ml_kem_k k);

Functions 1 through 9 constitute the API and are described in detail in the relevant document covering the interface and its usage.

- void ml_kem_decaps_ct_select_ss(). This function performs the cryptographic part of decapsulation and operates on secrets; therefore, it is highly sensitive to timing and memory leaks.
- size_t get_s_vector() - Directly performs the packing of the secret vector s
- size_t unpack_s_vector() Performs the actual unpacking of the secret vector s

2.3 Module ml_kem_create_keys.c

This module is responsible for initializing the ML-KEM operation, its task is to generate keys and return them as a structure of type ml_kem_ctx, where the keys and a set of persistent data required for calculation are stored.

The ml_kem_temp structure is also used here, temporary data is stored in the fields of this structure, which are not needed after the generation of ready-made keys. Therefore, a temporary structure is introduced and use it during generation and zeroing and destruction at the end of key generation.

Before describing all the functions of this module, for convenience, I recommend opening diagrams 4.3a and 4.3b - these are general diagrams of sequential execution of key generation, diagram 4.4 is a clear schematic representation of the use of existing functions in the module.

The main idea is:

1. Calling two allocation functions for two structures of type ml_kem_ctx and ml_kem_temp, where ml_kem_temp is a temporary structure that is needed only during the key generation process, and ml_kem_ctx is the final result — it contains the keys and additional parameters that are needed during the decapsulation process
2. Calling the main key generation function, it is described in detail in subsection 3.3.1. It is this function that controls everything and from it, during the key generation process, other functions are called and and the call stack expands accordingly.
3. After the main key generation function has completed, the destructor for the temporary structure should be called. It is advisable not to postpone this operation, since after the main key generation function has completed, the temporary structure is no longer needed.
4. The persistent structure in which keys and other important values are stored is passed on for work that goes beyond the scope of this module. It should only be noted that in the case when the keys are no longer needed, the destructor of the persistent structure should be called - it is also located in this module.

This module has the following functions that are used in other modules of this project:

2.3.1. int ml_kem_create_ctx_struct(struct ml_kem_temp *temp, struct ml_kem_ctx *ctx, ml_kem_entropy_fn entropy)

int ml_kem_create_ctx_struct(struct ml_kem_temp *temp, struct ml_kem_ctx *ctx) is the main key generation function, it is in it that most other static functions that work for key generation are called, and it fills the ml_kem_ctx and ml_kem_temp structures and operates on them when generating keys. It takes three pointers as parameters — of type ml_kem_temp to a temporary structure, ml_kem_ctx to a persistent structure and entropy function. It works in the following sequence:

1. Validation of data and structure members responsible for the values of certain fields (initial data is set during allocation)
2. Generating entropy using ml_kem_generation_entropy() and writing them to a temporary structure
3. Creating a private key using ml_kem_create_vector_poly() into a temporary structure
4. Creating a public key using ml_kem_create_public_key() into a temporary structure
5. Forming the final byte-encoded public key message used by encapsulation to generate ciphertext, and copying all persistent values into a persistent structure
6. Creating z (as a fallback value in case of ciphertext mismatch of ciphertexts during re-encapsulation in decapsulation) and creating a hash from the open message and writing all this into the corresponding fields of the persistent structure, all this is done in ml_kem_finally_create_z_and_h_pk()

   A critical remark regarding the operation of this function is that it only works and

operates with these pointers to persistent and temporary structures. It is obvious that they should be destroyed separately using destructors. The temporary structure is preferably destroyed immediately after this function has run, and the persistent structure is destroyed when the key is no longer needed.

2.3.2. struct ml_kem_temp *ml_kem_temp_alloc(enum ml_kem_k level)

struct ml_kem_temp *ml_kem_temp_alloc(enum ml_kem_k level) is an allocation function for a temporary structure, i.e. where temporary data is used, needed only for key generation. It takes only one parameter - the security level. It is executed in the following sequence:

1. Checking the correctness of the obtained security level and determining the parameters for key generation - η and the number of bytes with SHAKE for the polynomial
2. Allocation of all fields that have an unsigned two-byte type and distribution of the resulting memory between fields of this type
3. Allocation of all fields that have an unsigned single-byte type and distribution of the resulting memory between fields of this type
4. Returning the result

Note that a number of important parameters that determine the dimensionality of vectors, η, and the number of raw bytes required for polynomials are defined and recorded here — in the allocation function.

2.3.3. struct ml_kem_ctx *ml_kem_ctx_alloc(enum ml_kem_k level)

struct ml_kem_ctx *ml_kem_ctx_alloc(enum ml_kem_k level) is an allocation function for a persistent structure, this is a persistent structure that stores keys and other important data that lives until the keys are destroyed. It takes only one parameter - the security level. It is executed in the following sequence:

1. Checking the correctness of the obtained security level and determining the parameters for key generation - η and the number of bytes with SHAKE for the polynomial
2. Memory allocation for a secret
3. Memory allocation for the final public key message including the matrix seed
4. Returning the result

Note that a number of important parameters that determine the dimensionality of vectors, η, and the number of raw bytes required for polynomials are defined and recorded here — in the allocation function.

2.3.4. void ml_kem_destroy_temp_struct(struct ml_kem_temp *temp)

void ml_kem_destroy_temp_struct(struct ml_kem_temp *temp) — this function is a destructor for a temporary structure, all data that was allocated by the function from subsection 3.3.2. The data that was allocated in the allocator is sequentially zeroed and destroyed in relation to the number of allocator calls and initial pointers to memory areas.

2.3.5. void ml_kem_destroy_ctx_struct(struct ml_kem_ctx *ctx)

void ml_kem_destroy_ctx_struct(struct ml_kem_ctx *ctx) - this function is a destructor for a persistent structure, all data that was allocated by the function from subsection 3.3.3. It works similarly to the destructor from subsection 3.3.4.: the data that was allocated in the allocator is sequentially zeroed and destroyed in relation to the number of allocator calls and initial pointers to the memory area.

This module has the following functions that are used only within this module:

2.3.6. int ml_kem_generation_entropy(struct ml_kem_temp *temp, ml_kem_entropy_fn entropy)

This function generates primary seeds, from which buffers with bytes are then obtained via SHAKE for further key calculation, accepts a pointer to a temporary structure, returns an integer value regarding the success of the seed generation operation. The execution sequence is as follows:

1. Generating random 32 bytes via a reliable entropy source. This source is either taken from code in the system or passed as a parameter
2. Adding to the received 32 bytes (concatenation) the 33rd byte — which is the value of the vector dimension (2, 3 or 4)
3. Obtaining from the available 33 bytes via SHA3-512 two seeds for the matrix and noise/secret vectors
4. Copying the two received values in the previous sub-item into the corresponding fields of the structure

2.3.7. void ml_kem_create_vector_poly(struct ml_kem_temp *temp)

This function creates a secret vector and converts it from coefficient to point format. It takes a pointer to a temporary structure. It works in the following sequence:

1. Copies the seed for the secret and noise vectors, and adds a counter byte to it at the end (concatenates)
2. In the loop, the last byte of the seed is counted as a counter
3. Calls SHAKE256 passing the seed of the current iteration and receives a stream of bytes, the amount of bytes received depends on the parameter η
4. Calls void ml_kem_gen_polynomial_for_cbd(u16 *poly, const u8 *stream, const u8 eta) to obtain already complete polynomials
5. Using the functions of the ml_kem_ntt_main.c module, it converts a polynomial into a point domain

Thus, in the loop, the required number of polynomials for the vector are created using the vector dimension value stored in the temporary structure.

2.3.8. int ml_kem_create_public_key(struct ml_kem_temp *temp)

This function creates a public key. It takes a temporary structure as a parameter. It is executed in the following sequence:

1. Setting up counters for the noise seeds and matrix A and concatenating the counters (last byte for the array) with each of the seeds at the end of the array
2. Running loops that generate polynomials from seed using int ml_kem_gen_polynomial_for_matrix(u16 *poly, const u8 *stream, int suma_bytes) — on each iteration one polynomial in strict correspondence for the matrix

3. Multiply the generated matrix polynomial by the corresponding polynomial of the secret vector according to the public key calculation formula
4. Accumulation of the result. In this way, the scalar product of a certain row of the matrix and the secret vector is formed
5. Generating a noise byte stream for a string via SHAKE256 from a seed
6. Calls void ml_kem_gen_polynomial_for_cbd(u16 *poly, const u8 *stream, const u8 eta) to obtain a fully-fledged noise polynomial
7. Converting the noise polynomial to a point format and adding it to the scalar product of the matrix row and the secret vector, thereby forming a full-fledged polynomial of the public key vector.

This operation is performed in cycles, it should be noted that the ml_kem_gen_polynomial_for_matrix() function, theoretically, can return -EAGAIN, this is if the generated buffer does not have enough bytes for the matrix polynomial, the probability of this is abnormally small, but if this happens, then you should simply start key generation again, from the very beginning.

2.3.9. void ml_kem_convert_to_bytes_format_and_copy(struct ml_kem_temp *temp, struct ml_kem_ctx *ctx)

This function converts the public key in the final message into a single-byte format and adds the matrix seed to it. This operation is performed in a loop, where each byte-wise coefficient is extracted from a two-byte array and, according to the standard, is written to a single-byte array, at the end of this loop, 32 bytes of the matrix seed A are concatenated.

2.3.10.int ml_kem_finally_create_z_and_h_pk(struct ml_kem_ctx *ctx, struct ml_kem_temp *temp, ml_kem_entropy_fn entropy);

This function generates a seed for secret *z*, this seed is needed in decapsulation, in case the ciphertexts do not match. This seed is concatenated with the hash of the received ciphertext and is also hashed, and the resulting 32 bytes are sent in the case specified in the previous sentence during the decapsulation process.

Also, the hash of the public key is calculated and written to the permanent key structure here - it is needed during encapsulation, and in order to avoid a permanent hash in decapsulation - it is better to do it once and every time during encapsulation, which is part of decapsulation, just read this hash and that's it.

2.4. ml_kem_encaps.c

This module implements the encapsulation operation in the ML-KEM algorithm. It should be noted that encapsulation is performed both for generating and sending a message and as part of the decapsulation process, therefore, two modes are provided for performing this operation:

1. Standard encapsulation for message formation (hereinafter referred to as message creation mode).
2. Encapsulation as part of decapsulation (hereinafter referred to as decapsulation mode)

Therefore, when describing this module, this module description highlights the difference in execution modes and application of its functions.

The main idea is:

1. Allocation of a structure for encapsulation (ml_kem_encaps_ctx) in message creation mode or selection of a free slot from the pool (where the required encapsulation structure is) in decapsulation mode.

2. Running the main encapsulation function - void ml_kem_encapsulation(struct ml_kem_encaps_ctx *ctx, u8 *msg, u8 *hash_pk, ml_kem_entropy_fn entropy)
3. Reading the ciphertext and shared secret from the corresponding fields of the ml_kem_encaps_ctx structure - ciphertext and shared secret
4. Calling destructors in message creation mode, zeroing structures and freeing a slot in decapsulation mode

Before describing the functions, it is recommended to familiarize yourself with diagrams 4.5a, 4.5b, and 4.6, which illustrate the general encapsulation process and internal calculations.

The **ml_kem_encaps.c** module has the following functions that are used in other files:

2.4.1. struct ml_kem_encaps_ctx *ml_kem_alloc_encaps(u8 *public_key_msg, enum ml_kem_k k)

The allocation function is used in message creation mode, in decapsulation mode it is not needed, since the pool module allocates everything itself.

Receives the following parameters:

1. Single-byte public key message, according to the ML-KEM standard
2. Security level

Returns a pointer of type ml_kem_encaps_ctx.

The sequence of execution of this function is as follows:

1. Verification of received data
2. Allocation of fields of the ml_kem_encaps_ctx structure that have a two-byte type
3. Memory allocation for ciphertext

4. Memory allocation for the public key array (what was passed in the first parameter is copied into this allocated memory)
5. Distributing the allocated memory across structure fields and returning a pointer to structures

2.4.2. void ml_kem_wipe_encaps(struct ml_kem_encaps_ctx *ctx)

This is a function whose purpose is to zero out the fields of the encapsulation structure. It is called in decapsulation mode to make the slot clear for the new ciphertext.

2.4.3. void ml_kem_destroy_encaps(struct ml_kem_encaps_ctx *ctx)

Destructor — frees memory, needed for message creation mode. It is advisable to call the function from 3.4.2 before the destructor.

2.4.4. void ml_kem_encapsulation(struct ml_kem_encaps_ctx *ctx, u8 *msg, u8 *hash_pk, ml_kem_entropy_fn entropy)

The main encapsulation function — directly performs encapsulation. It accepts the following parameters:

1. Pointer to the structure for encapsulation ml_kem_encaps_ctx.
2. In decapsulation mode, the seed for encapsulation is accepted as the second parameter, if the message creation mode then NULL should be passed.
3. In decapsulation mode, the third parameter is the public key hash, if the message creation mode then NULL should passed.
4. Entropy function when needed

It is important to note that the first parameter is always passed, and the 2nd and 3rd only in case of encapsulation in decapsulation mode, but if it is encapsulation, as in creating a message, then both parameters (2nd and 3rd) must be NULL. This is stated to note that in the case when parameter 2 is passed when parameter 3 is NULL, or vice versa, then this is a mistake in the application of this function.

This function operates as follows:

1. Verification of received data
2. Mode definition: if the decapsulation mode is selected, then the 2nd and 3rd parameters are copied to the function's internal buffers, if not, then the seed is generated inside this function and the public key is also hashed.
3. Obtaining and storing the shared secret in the appropriate field of the encapsulation structure
4. Unpacking a public key using ml_kem_unpack_pk_t_u16()
5. Getting the value of the temporary secret vector in the NTT domain using ml_kem_create_temp_secret_y()
6. Calculating the value of *u(vector of polynomials)* in encapsulation using ml_kem_create_u()
7. Calculating the value of *v(compressed polynomial)* in encapsulation using ml_kem_create_v()
8. Compression and formation of the final message in the ml_kem_compress_ciphertext() function

As a result, the fields of the structure passed by the pointer will be filled with the necessary values (ciphertext and secret).

The ml_kem_encaps.c module has the following functions that are used only

within its scope:

2.4.5. void ml_kem_unpack_pk_t_u16(struct ml_kem_encaps_ctx *ctx)

The function unpacks the coefficients of the public key from a single-byte format to a 16-bit format. It accepts a pointer to the encapsulation structure, accesses the required fields, reads the single-byte array bit by bit, writes the obtained coefficients to a two-byte array, and writes them to the corresponding field of the structure according to the pointer that was passed.

2.4.6. void ml_kem_create_temp_secret_y(struct ml_kem_encaps_ctx *ctx, u8 *seed)

Function to calculate *ephemeral secret vector*. This vector is deterministically generated from the seed obtained during encapsulation using the SHAKE256 function, followed by converting the raw bytes into polynomials according to t*he centered binomial distribution (CBD)*. The resulting polynomials are converted to the NTT domain. The function is implemented according to the ML-KEM standard.

2.4.7. int ml_kem_create_u(struct ml_kem_encaps_ctx *ctx, u8 *seed)

The function for calculating the ciphertext *vector u* component during encapsulation. This vector is calculated as the product of the *transposed matrix A*, deterministically generated from the seed ρ using SHAKE128 and *rejection sampling*, by

the *ephemeral secret vector y*, with the subsequent addition of the noise vector e1, generated using SHAKE256 and *centered binomial distribution (CBD)*. The calculations are partially performed in the NTT domain for efficiency, after which the result is converted to coefficient form. The function is implemented according to the ML-KEM standard using the formula $u = A^T \cdot y + e1$.

2.4.8. void ml_kem_create_v(struct ml_kem_encaps_ctx *ctx, u8 *seed)

The function for calculating the *ciphertext component polynomial v* during the encapsulation process. This polynomial is calculated as the scalar product of the *public key vector t* and *the ephemeral secret vector y in NTT domain*, followed by converting the result to coefficient form. The resulting value is added to the *noise polynomial e2*, generated from the seed using SHAKE256 and the *centered binomial distribution (CBD)*, and the *message m encoded* in the polynomial representation. The function is implemented according to the ML-KEM standard by the formula $v = t^T \cdot y + e2 + m$.

2.4.9. void ml_kem_compress_ciphertext(struct ml_kem_encaps_ctx *ctx)

This function compresses *vectors u and v* into ciphertext. According to the standard, it determines compression parameters based on the security level and compresses vectors one polynomial at a time, writing them into a one-byte array. To compress one polynomial in a loop, it calls another function — ml_kem_compress_and_pack().

2.4.10. void ml_kem_compress_and_pack(u8 *out, const u16 *coeffs, size_t n, u8 d)

Compresses and packs a polynomial into a ciphertext byte stream. It takes the following parameters:

1. Output buffer (must be of sufficient size)
2. polynomial coefficients *(mod q)*
3. number of coefficients *(ML_KEM_N = 256)*
4. number of bits after compression

The function of compression and packing of polynomial coefficients. Each coefficient is compressed to *d bits* using the *Compress algorithm*, after which the results are sequentially packed into a byte array by bit accumulation. The formation of the output array is performed according to the ML-KEM specification.

2.4.11. u16 ml_kem_compress(u16 x, u8 d)

The coefficient compression function according to ML-KEM. It maps the value from *Zq* to a d-bit representation by scaling, rounding, and reducing, implementing the Compress operation without using division.

2.4.12. void ml_kem_create_u_full_matrix(struct ml_kem_encaps_ctx *ctx, u8 *seed);

If the #if FULL_MATRIX directive is set to 1, this function operates with a fully expanded matrix within an encapsulation that is executed as part of the decapsulation process.

2.5. ml_kem_decrypt.c

This module performs the decryption operation, which is part of the decapsulation. A special structure ml_kem_decrypt_ctx is introduced for this operation. This module decrypts the received ciphertext and recovers the *message m*. This module does not implement allocation or destruction functions; only a zeroization function is provided. These responsibilities are handled by the pool creation module. Before describing the module functions, it is recommended to familiarize yourself with diagrams 4.7. and 4.8., which reflect the general decryption process and the internal mathematical logic of the calculations.

The ml_kem_decrypt.c module has the following functions that are used in other files:

2.5.1. void ml_kem_decrypt(struct ml_kem_decrypt_ctx *ctx_decry, u8 *ciphertext)

This function performs the basic logic of the decryption formula in ML-KEM: $v - s^T \cdot u$.

Accepts the following parameters:

1. Pointer to the decryption structure ml_kem_decrypt_ctx
2. Pointer to a byte array containing the ciphertext

It does not return anything, it just fills the corresponding field in the ml_kem_decrypt_ctx structure with the received seed generated by the encapsulation. The sequence of execution of this function is as follows:

1. Copying the ciphertext into the corresponding field of the structure

2. Decompression using the function ml_kem_decompress()
3. Decryption in ml_kem_decrypt_get_poly() and getting the decryption result as a polynomial
4. Converting the resulting decrypted polynomial to 32-byte form using ml_kem_decrypt_get_seed_m()

2.5.2. void ml_kem_wipe_decrypt(struct ml_kem_decrypt_ctx *ctx_decry)

The zeroization function, its purpose is to nullify the sensitive fields of the ml_kem_decrypt_ctx decryption structure, except for those fields that are informative about the types and parameters of data, the security level at which the current structure operates, and a reference to the key structure itself.

The ml_kem_decrypt.c module has the following functions that are used only within its scope:

2.5.3 void ml_kem_decrypt_get_poly(struct ml_kem_decrypt_ctx *ctx_decry)

This function performs the decryption itself, working with the polynomials already unpacked from the message, which are written in the fields of the ml_kem_decrypt_ctx structure. It takes only one parameter - a pointer to the decryption structure. It does not return anything, but writes the result of the operation to the corresponding field of the ml_kem_decrypt_ctx structure. It works in the following sequence:

1. Performs scalar multiplication of the polynomial *vectors u and s^T*
2. Converts the result to a coefficient representation
3. Starts a cycle of subtracting *polynomials v* and the resulting scalar *multiplication*

result u and s^T

2.5.4 void ml_kem_decrypt_get_seed_m(struct ml_kem_decrypt_ctx *ctx_decry)

This function performs a loop operation to convert the result from the polynomial format to the 32-byte value format, which is the seed generated by the encapsulation. For each bit in the loop, ml_kem_decode1_bit() is called.

2.5.5. void ml_kem_decompress(struct ml_kem_decrypt_ctx *ctx_decry)

The function for decompressing the ciphertext components. Performs sequential recovery of *the vector of polynomials u and the polynomial v* from the compressed byte array according to the ML-KEM parameters. For each polynomial, *the Decompress operation is applied with the parameters du and dv*, which are determined by the security level, and is performed using the ml_kem_decompress_one_poly() function. The result is the coefficients of the polynomials in *Zq* for further decapsulation.

2.5.6. void ml_kem_decompress_one_poly(u16 *poly, u8 *compress_poly, const u8 d)

Decompress function for a single polynomial. Recovers the coefficients of a polynomial from a bit-packed representation by extracting d-bit values sequentially from the byte stream. For each coefficient, a Decompress operation is applied, which recovers the values in *Zq*. Processing is performed through a bit buffer for correct alignment and

sequential reading of the data according to the ML-KEM specification.

2.5.7. u16 ml_kem_decompress_one_coeff(u16 y, u8 d_u)

Function that performs the operation according to the decompression formula in ML-KEM decoding of one coefficient for *u or v.*

2.5.8. u8 ml_kem_decode1_bit(u16 w)

Function for decoding a single bit of a message from a polynomial coefficient. Determines the bit value based on whether the coefficient falls within the interval corresponding to an encoded 1, using threshold values associated with the modulus q. Implements the Decode operation according to ML-KEM.

2.6. ml_kem_decaps_pool.c

Before describing the functions, it is recommended to familiarize yourself with diagrams 4.9., 4.10., 4.11.,4.12., which show the structure of the pool, its internal logic and its application. This module implements a pool of slots for decapsulation. Each slot contains two structures:

- ml_kem_decrypt_ctx – for decryption
- ml_kem_encaps_ctx – for encapsulation

Both of these structures are used together in the decapsulation process.

The main idea of the architecture is to divide data into:

- persistent — not changing between calls

- temporary — used for a single ciphertext

Persistent data is stored in the structure:

- ml_kem_ctx — contains keys and security level parameters

Temporary data is stored in:

- ml_kem_decrypt_ctx
- ml_kem_encaps_ctx

These structures are used only when processing a single ciphertext and can be zeroized after use, except for fields containing public or service data (e.g., references to ml_kem_ctx or level parameters). Thus:

1. ml_kem_ctx exists throughout the key lifecycle
2. pairs (decrypt_ctx, encaps_ctx) from the pool slots

Each slot:

- has the same structure and size (determined by the security level)
- is associated with a specific ml_kem_ctx
- can be used to process any ciphertext

During decapsulation:

1. A thread atomically grabs a free slot
2. Performs decapsulation
3. Zeroizes temporary data
4. Frees up a slot

Slot access is controlled via *the atomic flag* is_free. The number of slots is set during initialization and depends on the allowable level of parallelism in the system.

The point of the architecture is that we allocate more memory once to avoid frequent allocations in the future. This avoids frequent allocations during decapsulation, while maintaining the possibility of parallel processing without introducing a bottleneck. The only requirement is to allocate significantly more memory during key generation,

depending on what the environment where ML-KEM will run in a given implementation can afford.

The functions of this module are accessed during the key generation process, as soon as the filled ml_kem_ctx structure is received.

The **ml_kem_pool.c** module has the following functions that are used in other files:

2.6.1. struct ml_kem_general_keys_decaps_ctx *ml_kem_alloc_decaps_pool(enum ml_kem_k level, size_t ml_kem_pool_count, struct ml_kem_ctx *private_keys);

This function performs the main logic of this module. It allocates all slots and memory for them, and also calls the pool head allocation function. It accepts the following parameters:

1. Security level
2. Number of slots
3. Pointer to the ml_kem_ctx structure — a ready-made structure of keys and persistent data

   Returns a pointer to the pool head — ml_kem_pool_decaps_ctx.

   It works in the following sequence:

1. Check the received parameters
2. Derives byte pointers for the public key and ciphertext (depending on the security level)
3. Pool head allocation ml_kem_alloc_head_decaps_pool()
4. Assigns length values for the ciphertext and public key in the pool head structure
5. Allocation of all encapsulation structures

6. Allocation of all decryption structures
7. Memory allocation for ciphertext pointers for all encapsulation and decapsulation structures
8. Memory allocation for public key pointers for all encapsulation structures
9. Memory allocation for raw byte pointers for all encapsulation structures
10. Computes the total number of u16 pointer fields across all encapsulation and decryption structures and allocates memory for them
11. Running a loop that allocates memory across all pointers in all slots and at the head of the pool using the ml_kem_get_mem_two_bytes_ptrs_encaps() and ml_kem_get_mem_two_bytes_ptrs_decrypt() functions
12. Return result — pointer of type ml_kem_pool_decaps_ctx.

2.6.2. void ml_kem_pool_destroy(struct ml_kem_general_keys_decaps_ctx *head_pool)

This function acts as the destructor for the pool created in subsection 3.6.1. It should be noted that it does NOT change the ml_kem_ctx structure, but destroys only what is allocated specifically for the pool and slots.

The **ml_kem_pool.c** module has the following functions that are used only within its scope:

2.6.3. struct ml_kem_general_keys_decaps_ctx *ml_kem_alloc_head_decaps_pool(size_t ml_kem_pool_count)

Performs the function of creating a pool head - the main structure that points to a set of slot structures. Takes one parameter - the number of slots. Returns a pointer to the

pool head of type ml_kem_pool_decaps_ctx. Works in the following sequence:

1. Pool head structure allocation
2. Slot allocation
3. Fills all pool header fields with NULL, pointing specifically to the memory areas that have been allocated to all slots
4. Initializes all slot pointers to NULL

2.6.4. u16 *ml_kem_get_mem_two_bytes_ptrs_encaps(struct ml_kem_encaps_ctx *ctx, u16 *two_bytes_ptrs, enum ml_kem_k level)

A memory allocation function for all fields of two-byte pointers of a specific encapsulation structure. Accepts:

1. Pointer to this structure
2. The memory area itself, which is allocated by pointers to the structure fields
3. Security level

The function returns a pointer to the same allocated memory area, but shifted by a certain offset. The skipped part of the memory is used to store the internal pointer structures.

2.6.5. u16 *ml_kem_get_mem_two_bytes_ptrs_decrypt(struct ml_kem_decrypt_ctx *ctx, u16 *two_bytes_ptrs, enum ml_kem_k level)

A memory allocation function for all fields of two-byte pointers of a specific decryption structure. Accepts:

1. Pointer to this structure
2. The memory area itself, which is allocated by pointers to the structure fields

3. Security level

The function returns a pointer to the same allocated memory area, but shifted by a certain offset. The skipped part of the memory is used to store the internal pointer structures.

2.6.6. u16 *get_full_matrix(struct ml_kem_ctx *ctx_keys, struct ml_kem_general_keys_decaps_ctx *ctx_global);

If the #if FULL_MATRI directive is set to 1, this function fully expands the matrix within an encapsulation that is executed as part of the decapsulation.

2.7. ml_kem_ntt_main.c

This module implements operations on polynomials in the NTT domain (Number-Theoretic Transform), which are key to the efficient operation of the ML-KEM algorithm. The main goal of the module:

1. Converting polynomials to NTT domain
2. Performing the inverse transformation
3. Implementing multiplication and addition in the NTT domain

The implementation fully complies with the algorithms defined in the standard (section 4.3 The Number-Theoretic Transform, algorithms 9–12).

In ML-KEM, polynomials belong to the ring:

$$R_q = \mathbb{Z}_q[X]/(X^n + 1)$$

, where

- q=3329
- n=256

NTT is used to replace slow polynomial multiplication with fast coordinate multiplication in transformed space.

- Ordinary domain → ordinary polynomial multiplication
- NTT-domain → pair-wise multiplication

2.7.1. Constant tables

ml_kem_zetas. The ml_kem_zetas array contains 128 pre-computed twiddle factors and is used in the NTT and iNTT algorithms. These values are:

- clearly defined by the standard (Appendix A)
- correspond to the roots of unity in the field Zq

ml_kem_gammas. The ml_kem_gammas array is used to implement BaseCaseMultiply and corresponds to the coefficients γ from the multiplication algorithm in the NTT domain (Algorithm 12 in the standard).

The main functions of the module are as follows:

2.7.2. void ml_kem_ntt_fips203(u16 f[ML_KEM_N])

Performs direct NTT transformation of a polynomial. Required to convert a polynomial from coefficient form to NTT domain. Implementation features:

1. Implements Algorithm 9 (NTT) from the standard
2. Uses Cooley–Tukey butterfly

3. Works in-place (without additional memory)

The main idea is that at each step:

1. A pair of coefficients is taken
2. The “butterfly” is used:

```
u = f[j]
t = zeta * f[j + len]
f[j] = u + t
f[j + len] = u - t
```

The traversal order and indexing of zetas clearly conform to the standard.

2.7.3. void ml_kem_intt_fips203(u16 f[ML_KEM_N])

Performs the inverse NTT transformation, i.e. converts a polynomial from the NTT domain back to coefficient form. Implements Algorithm 10 from the standard and uses the Gentleman–Sande butterfly.

At the end, normalization is necessarily performed:

```
f[i] = f[i] * n^{-1} mod q,
```

where

ML_KEM_COEFF_NORMALIZATION = *3303*, and it is the inverse of *n=256* in the *Zq* field.

2.7.4. void ml_kem_multiply(u16 result[ML_KEM_N], u16 first[ML_KEM_N], u16 second[ML_KEM_N])

Multiplies two polynomials in the NTT domain. Compliant with the standard — algorithms 11(MultiplyNTTs) and 12(BaseCaseMultiply). The idea is to represent the polynomial as 128 pairs of coefficients, for each pair:

$$(a_0 + a_1 \cdot X) * (b_0 + b_1 \cdot X)$$

Calculated as:

$$c_0 = a_0 * b_0 + \gamma * a_1 * b_1$$

$$c_1 = a_0 * b_1 + a_1 * b_0$$

where γ is taken from ml_kem_gammas.

This is not ordinary multiplication - it is a special structure of the NTT domain.

2.7.5. void ml_kem_add(u16 result[ML_KEM_N], u16 first[ML_KEM_N], u16 second[ML_KEM_N])

Performs coordinate-wise addition of polynomials in the NTT domain. Corresponds to the usual addition operation in Tq. Implemented via:

result[i] = first[i] + second[i] mod q

2.7.6. Interaction with other modules

The module uses(implemented in the arithmetic module):

1. ml_kem_add_mod
2. ml_kem_sub_mod
3. ml_kem_multipl_mod

Used in:

1. key generation
2. encapsulation
3. decryption

Key implementation features:

1. Fully FIPS 203 compliant
2. Uses only integer arithmetic (no float — a standard requirement)
3. In-place processing (minimum memory)
4. Fixed tables (without runtime generation)

Note.

The operations of the butterfly algorithm are performed in a specialized variation of this algorithm. This variation is not the same as the "classic butterfly" and is designed specifically for the ML-KEM algorithm.

2.8. ml_kem_shake.c

This module implements wrappers around SHAKE128 and SHAKE256 based on the Keccak-f[1600] sponge construction, which are used in ML-KEM as an XOF (extendable-output function) according to the standard.

The main task of the module is to generate pseudo-random bytes for:

1. matrix A expansion
2. noise generation
3. derivation of secret vectors

The module implements the classic sponge model through the ml_kem_sponge structure:

1. Init — initialize state
2. Absorb — absorbing input data into the state
3. Finalize — padding and final Keccak permutation
4. Squeeze — extract an arbitrary number of bytes

Based on this, two public functions are provided:

1. ml_kem_shake128 (rate = 168 bytes)
2. ml_kem_shake256 (rate = 136 bytes)

The main implementation features:

- fully constant-time (no dependence on secret data)
- no secret-dependent branching and division/modulus operations on secrets
- all memory access is based only on public indices (pos)
- the state is securely zeroized after use

In ML-KEM, this module is a low-level primitive for deterministic byte stream generation from seeds, used in key generation, encapsulation, and the NTT pipeline.

2.9. ml_kem_sha3.c

This module implements local hash functions SHA3-256 and SHA3-512 based on the Keccak-f[1600] sponge construction, which are used in ML-KEM as cryptographic hash functions according to the standard.

The module provides:

- computation of SHA3-256 (32 bytes)
- computation of SHA3-512 (64 bytes)

It is used for:

- hashing public keys
- seed derivation
- construction of cryptographic primitives (G, H in ML-KEM)

The implementation is based on the ml_kem_sha3_ctx structure, which contains:

- Keccak state (1600 bits)
- rate parameter (136 or 72 bytes)
- position within the block

The module implements the classical sponge sequence:

1. Init — state initialization and zeroing
2. Absorb — absorbing input data into the state (XOR into state)
3. Finalize — padding (pad10*1 + domain separation 0x06)
4. Squeeze — extraction of a fixed-length hash

Public functions:

- ml_kem_sha3_256()
- ml_kem_sha3_512()

Both functions:

- perform the full sponge cycle
- return a fixed-length hash
- securely clear the state after completion

In the ML-KEM architecture, this module is a fundamental cryptographic component that implements fixed-length hash functions (not XOF) and is used together with the SHAKE module: SHA3 for hashing and SHAKE for byte stream generation.

2.10. keccak_1600.c

This module implements the Keccak-f[1600] permutation (24 rounds) — the core function on which SHA3 and SHAKE are built. It performs an in-place transformation of a 1600-bit state (25 × u64).

The function keccak_f1600_ct(state) performs 24 rounds consisting of:

1. θ (Theta) — column mixing
2. ρ + π (Rho + Pi) — rotation and permutation
3. χ (Chi) — non-linearity
4. ι (Iota) — addition of the round constant

This function serves as the core of SHA3-256, SHA3-512, SHAKE128, and SHAKE256, which are implemented in the corresponding modules.

2.11. ml_kem_encaps_pool.c

The encapsulation pool management module is responsible for allocating, initializing, and organizing reusable memory resources for ML-KEM encapsulation operations.

The module supports ML-KEM-512, ML-KEM-768, and ML-KEM-1024, allowing independent pools of encapsulation slots to be configured for each security level. Each slot contains a dedicated encapsulation context and its own working buffers, enabling concurrent operations without repeated memory allocation.

Memory is allocated in contiguous blocks and divided among individual contexts to reduce allocation overhead, minimize memory fragmentation, and improve memory locality.

The module consists of three functions:

2.11.1. ml_kem_create_all_structs()

Allocates and initializes the encapsulation slot descriptors and their associated `ml_kem_encaps_ctx` structures for the specified ML-KEM security level.

The function allocates two contiguous arrays: one containing the slot descriptors and another containing the encapsulation contexts. Each slot is linked to its corresponding context, and its atomic availability flag is initialized.

Additionally, the function configures the security level, public key length, and ciphertext length for each context and registers the allocated structures within the appropriate section of the encapsulation pool.

2.11.2 ml_kem_assign_encaps_pool_u16_buffers()

Allocates and assigns the 16-bit polynomial working memory required by all encapsulation contexts of a specified security level.

The function allocates a single contiguous memory block and divides it into independent regions for each slot. These regions contain the unpacked public key polynomial vector, ciphertext components `u` and `v`, ephemeral secret vector `y`, and temporary polynomial buffer.

The allocated memory is zero-initialized, and the corresponding pointers are assigned to each encapsulation context. The original allocation pointer is preserved in the pool structure for subsequent cleanup.

2.11.3. ml_kem_assign_encaps_pool_u8_buffers()

Allocates and assigns byte-oriented working memory for all encapsulation contexts of a specified security level.

The function creates a single contiguous memory block containing the serialized public key buffers, ciphertext buffers, and temporary XOF output buffers required for polynomial generation.

The allocation size depends on the selected ML-KEM security level and the number of slots. The memory is zero-initialized and divided into independent regions assigned to individual encapsulation contexts.

The original allocation pointer is stored in the pool structure for subsequent memory management and destruction.

2.12. Structure Description

In this implementation, all structures represent a grouping of logically related data and/or data required to perform a specific high-level operation (for example, encapsulation has its own dedicated structure).

The implementation includes the following set of structures:

1. ml_kem_ctx — a structure that stores keys and other persistent data required throughout the entire lifecycle of the keys.
2. ml_kem_temp — a temporary structure used only during the key generation process. It contains fields that are required exclusively for key generation. This structure should be destroyed after the final key generation result is obtained.
3. ml_kem_encaps_ctx — a structure that contains the set of fields required for encapsulation.
4. ml_kem_decrypt_ctx — a structure that contains the set of fields required for the decryption process.
5. ml_kem_decaps_ctx — a structure that contains the set of fields required for the decapsulation process.

6. struct ml_kem_general_keys_decaps_ctx — a structure that serves as the head of the pool and manages all memory used in the decapsulation process.
7. struct ml_kem_encaps_slot - structure for a single encapsulation pool slot
8. struct ml_kem_pool_encaps_ctx - structure for the encapsulation pool head

A detailed description of each structure is provided in the following subsections.

2.12.1. ml_kem_ctx

This structure has the following representation in C:

```
struct ml_kem_ctx {
  u8 seed_rho[ML_KEM_SEED_BYTES];
  enum ml_kem_k k;
  u16 *secret;
  u8 *public_key_msg;
  size_t public_key_msg_len;
  u8 z[ML_KEM_SEED_BYTES];
  u8 h_pk[ML_KEM_SEED_BYTES];
};
```

Field description:

1. u8 seed_rho[ML_KEM_SEED_BYTES] — a seed (initial 32 bytes) *for matrix A*. During encapsulation and key generation, this seed is used to derive the full *matrix A*, which is required for computations.
2. enum ml_kem_k k — the security level. It is used for validation and parameter selection and remains constant throughout the entire lifetime of the key.
3. *u16 secret — the secret *vector s*, required only during the decryption process. In this implementation, it is stored in a two-byte (u16) polynomial format, which is

convenient for computations. No alternative representation is provided, since this data is secret, used only internally for calculations, never transmitted, and destroyed when the keys are no longer needed.

4. *u8 public_key_msg — the public key in byte format, represented as a ready-to-use message according to the ML-KEM standard.
5. size_t public_key_msg_len — the size of the public key, depending on the selected security level.
6. u8 z[ML_KEM_SEED_BYTES] — *a secret value* $\boldsymbol{z}$, used during decapsulation.
   If, after re-encapsulation, the ciphertexts do not match, this value is used to derive and return a fallback ("fake") shared secret that is indistinguishable from a valid one.
7. u8 h_pk[ML_KEM_SEED_BYTES] — a hash of the public key.
   It is required during encapsulation for shared secret generation. This field is stored in the structure because it remains constant throughout the key lifetime and is required during re-encapsulation.

The description above reflects that all fields in this structure are persistent and immutable throughout the entire lifetime of the keys and are required during the decapsulation process (from the perspective of the party holding the secret key).

This structure is produced by the key generation module and, within this implementation, is effectively treated as a closed object that is only used during decapsulation. Destroying this structure is equivalent to destroying the keys.

2.12.2. ml_kem_temp

This structure has the following representation in C:

```
struct ml_kem_temp {
  u8 seed_sigma[ML_KEM_SEED_BYTES];
```

```
    u8 seed_rho[ML_KEM_SEED_BYTES];
    enum ml_kem_k k;
    u16 *poly;
    u16 *public_key;
    u16 *secret;
    u8 *raw_bytes;
    size_t number_raw_bytes_vectors;
    u8 eta;
};
```

Field description:

1. u8 seed_sigma[ML_KEM_SEED_BYTES] — a seed (initial bytes) for generating noise and secret vectors. Since this is sensitive data required only during the generation of the secret vectors, it is not stored in persistent structures.
2. u8 seed_rho[ML_KEM_SEED_BYTES] — a seed for *matrix A*. It is also required in persistent data and therefore is copied into ml_kem_ctx. It is included here for convenience, so that a set of key generation functions can operate using a single pointer to a single structure (ml_kem_temp), which helps avoid confusion during the key generation process.
3. enum ml_kem_k k — the security level. It is required not only in the persistent structure but also in the temporary one, since it is needed during key generation.
4. *u16 poly — a pointer to an array of 256 elements of type u16.

   It serves as a temporary container for a single polynomial used in intermediate computations and is frequently overwritten. It is used only for mathematical operations during key generation.
5. *u16 public_key — the public key in a two-byte (u16) format, convenient for computations, but not the final representation defined by the standard.

6. *u16 secret — the secret vector. This field stores the generated secret for convenience. It is also included in the temporary structure to keep the computation flow within a single structure pointer. The value is copied into ml_kem_ctx and must be immediately destroyed after copying.
7. *u8 raw_bytes — a buffer for a byte stream produced by SHAKE128 and SHAKE256 from initial seeds. It is used to generate bytes both *for matrix A* and for secret and noise vectors.
8. size_t number_raw_bytes_vectors — the size of the raw_bytes buffer.
9. u8 eta — the parameter η, required for *the CBD (Centered Binomial Distribution) algorithm*.

All listed fields are either temporary values required during intermediate mathematical and cryptographic computations in the key generation process, or results that are later copied into the final ml_kem_ctx structure. For convenience and to simplify the logic of the key generation module, these fields are grouped into a single structure, allowing functions to operate using only one structure pointer. Once all required results are obtained and copied into ml_kem_ctx, this structure must be immediately zeroized and destroyed, as it contains sensitive data. The main reason for introducing this structure is the inability to avoid temporary intermediate data during key generation, which requires dedicated storage and is not limited to a single function.

2.12.3. ml_kem_encaps_ctx

This structure has the following representation in C:

```
struct ml_kem_encaps_ctx {
  u8 seed_rho[ML_KEM_SEED_BYTES];
  u8 seed_m[ML_KEM_SEED_BYTES];
```

```
    u8 K_bar[ML_KEM_SEED_BYTES];
    enum ml_kem_k k;
    u16 *u;
    u16 *v;
    u16 *public_key;
    u16 *y;
    u16 *poly;
    u8 *public_key_msg;
    size_t public_key_msg_len;
    u8 *ciphertext;
    size_t ciphertext_len;
    u8 *raw_bytes;
    s16 err;
struct shake_ctx shake;
struct sha3_ctx  sha3;
u16 *full_matrix;
};
```

Field description:

1. u8 seed_rho[ML_KEM_SEED_BYTES] — seed for *matrix A*.
2. u8 seed_m[ML_KEM_SEED_BYTES] — buffer for the message being encapsulated and transmitted within the ciphertext.
3. u8 K_bar[ML_KEM_SEED_BYTES] — buffer for the derived shared secret.
4. enum ml_kem_k k — security level.
5. *u16 u — vector (in two-byte representation) used to encode and transmit the temporary *secret y*.
6. *u16 v — vector (in two-byte representation) used to encode and transmit the encapsulated 32-byte message.

7. *u16 public_key — public key in a two-byte vector format convenient for computations.
8. *u16 y — temporary secret used during encapsulation.
9. *u16 poly — temporary buffer used to store a single polynomial during intermediate mathematical computations.
10. *u8 public_key_msg — buffer storing the public key in its serialized (message) format.
11. size_t public_key_msg_len — size of the serialized public key.
12. *u8 ciphertext — buffer containing the final ciphertext, which is transmitted as the output message.
13. size_t ciphertext_len — size of the ciphertext.
14. *u8 raw_bytes — buffer for byte streams generated by SHAKE128 and SHAKE256 during vector generation from seeds.
15. s16 err - variable for error code
16. struct shake_ctx shake and struct sha3_ctx sha3 - Required for the Keccak-1600 context; 200 bytes of allocated memory are provided separately for this purpose.
17. u16 *full_matrix; - A pointer to a fully unrolled matrix is included only if specified during compilation.

General description

All fields in this structure are required during the encapsulation process. They include both intermediate values and fields that form the final outputs. To avoid unnecessary architectural complexity, all related data is grouped into a single structure logically bound to one operation — encapsulation — which is performed frequently, unlike key generation. This structure contains all necessary fields for:

- standard encapsulation (message creation), and
- re-encapsulation as part of the decapsulation process.

It can be observed that this structure contains fields similar to those used in key generation and key storage structures. This is not data duplication, but a deliberate design decision. Since ml_kem_ctx and ml_kem_temp contain sensitive data, they are not logically tied to the encapsulation process. Therefore, in this implementation, there is no direct linkage between these structures. Even during decapsulation, the re-encapsulation step is performed after decryption, which naturally separates these operations and allows their results to be compared. As a result:

- each structure operates within its own domain,
- each module works only with the data it actually requires,
- and the implementation enforces a strict separation of responsibilities.

This design ensures that modules remain independent and are not aware of structures irrelevant to their operation, providing clear data boundaries and improving maintainability.

2.12.4. ml_kem_decrypt_ctx

This structure has the following representation in C:

```
struct ml_kem_decrypt_ctx {
  struct ml_kem_ctx *ctx;
  size_t ciphertext_len;
  u16 *u;
  u16 *v;
```

```
    u16 *poly;
    u8 *ciphertext;
    u8 m[ML_KEM_SEED_BYTES];
};
```

Field description:

1. *struct ml_kem_ctx ctx — a pointer to the structure containing keys and persistent data.
2. size_t ciphertext_len — the length of the ciphertext expected for this decapsulation operation.
3. *u16 u — vector (in two-byte representation) used to store the transmitted temporary *secret y*.
4. *u16 v — vector (in two-byte representation) containing the encapsulated 32-byte message.
5. *u16 poly — a temporary buffer used to store a single polynomial during intermediate mathematical computations.
6. *u8 ciphertext — buffer containing the received ciphertext.
7. u8 m[ML_KEM_SEED_BYTES] — buffer for the message recovered from the ciphertext (the encapsulated message).

General description

If we examine all fields of the ml_kem_decrypt_ctx structure, except for the pointer to ml_kem_ctx, the following becomes evident: all fields are temporary and contain data associated with a specific ciphertext. In contrast, the ml_kem_ctx structure referenced by the first field contains persistent data tied to the key lifecycle. For this reason, a clear separation was introduced:

- temporary data used only within a single decryption operation for a specific ciphertext
- persistent data associated with keys and reused across multiple operations.

Architectural notes

This separation is important for the design of this implementation. Some fields must be zeroized after each decapsulation operation because they contain sensitive or no longer needed data. However, they are not deallocated, since their size remains constant for a given key lifecycle and they can be reused for processing subsequent ciphertexts. This approach allows:

- euse of allocated memory,
- reduction of allocation overhead,
- and clearer separation between per-operation state and long-lived key material.

2.12.5. ml_kem_decaps_ctx

This structure has the following representation in C:

```
struct ml_kem_decaps_ctx {
  struct ml_kem_encaps_ctx *encaps_ctx;
  struct ml_kem_decrypt_ctx *decrypt_ctx;
  u8 buff_for_rate[136];
  ml_kem_atomic_t is_free;
```

};

Field description:

1. *struct ml_kem_encaps_ctx encaps_ctx — pointer to the encapsulation structure used during re-encapsulation, which is part of the decapsulation process.
2. *struct ml_kem_decrypt_ctx decrypt_ctx — pointer to the decryption structure used to recover the message from the ciphertext.
3. u8 buff_for_rate[136] — A buffer for the first Keccak-1600 operation, containing the concatenation of z and the beginning of the ciphertext. Subsequently, the shake128 absorption process takes place using this ciphertext buffer.
4. ml_kem_atomic_t is_free — an atomic variable controlling access to this structure.

General description

It should be noted that this is not just a decapsulation structure, but rather a representation of a single slot within a pool of slots. Since the pool is designed specifically for the decapsulation process, each slot serves two roles:

- as an element of the pool (resource management unit), and
- as a complete decapsulation context.

Architectural notes

This structure intentionally does not contain low-level or “fine-grained” fields required for direct mathematical computations. Instead, it contains pointers to the encapsulation structure, and the decryption structure, which are responsible for performing the underlying computational operations. In addition, it contains two important fields:

- atomic_int is_free — must always be checked before using the slot, to ensure that it is not currently occupied by another decapsulation operation processing a different ciphertext.

- buff_for_rate — a temporary buffer used for the first call to Keccak-1600 to generate a fallback (“dummy”) message in the event of a ciphertext mismatch.

Design rationale

The buff_for_rate field is deliberately placed at the decapsulation level, because it is inherently part of the decapsulation logic:

- it is not part of encapsulation (which only produces ciphertext),
- it is not part of decryption (which only recovers the encapsulated message),
- and it is not part of key storage (which contains only persistent data).

Instead, it belongs exactly to the decision and fallback stage of decapsulation, where the ciphertext is validated and the final shared secret is derived.

2.12.6. struct ml_kem_general_keys_decaps_ctx

This structure has the following representation in C:

```
struct ml_kem_general_keys_decaps_ctx {
    struct ml_kem_decaps_ctx *ml_kem_pool;
    size_t ml_kem_pool_count;
    size_t ciphertext_len;
    size_t public_key_msg_len;
    size_t two_bytes_ptrs_total;
    u16 *two_bytes_mem_ptrs;
    u8 *public_key_msgs_mem_ptr;
    u8 *ciphertexts_mem_ptr;
```

```
    u8 *raw_bytes_mem_ptr;
    struct ml_kem_decrypt_ctx *decrypt_mem_ptr;
    struct ml_kem_encaps_ctx *encaps_mem_ptr;
};
```

Field description:

1. *struct ml_kem_decaps_ctx ml_kem_pool — pointer to the first slot in the array of slots, each associated with the same key.
2. size_t ml_kem_pool_count — number of slots in the pool.
3. size_t ciphertext_len — ciphertext size for the given key.
4. size_t public_key_msg_len — size of the public key message for the given key.
5. size_t two_bytes_ptrs_total — total size of the allocated memory region for two-byte (u16) data, which is later divided among all slots in the pool.
6. *u16 two_bytes_mem_ptrs — pointer to the beginning of the allocated memory region for two-byte data, shared across all slots.
7. *u8 public_key_msgs_mem_ptr — pointer to the beginning of the allocated memory region (byte-based) used exclusively for public key messages across all slots.
8. *u8 ciphertexts_mem_ptr — pointer to the beginning of the allocated memory region (byte-based) used exclusively for ciphertexts across all slots.
9. *u8 raw_bytes_mem_ptr — pointer to the beginning of the allocated memory region (byte-based) used for byte streams and buffers produced by SHAKE128 and SHAKE256.
10. *struct ml_kem_decrypt_ctx decrypt_mem_ptr — pointer to the beginning of the allocated memory region containing an array of decryption context structures, one per slot.

11. *struct ml_kem_encaps_ctx encaps_mem_ptr — pointer to the beginning of the allocated memory region containing an array of encapsulation context structures, one per slot.

General description

This structure implements a pool of slots for decapsulation, which allows:

1. avoiding frequent memory allocations,
2. enabling parallel processing of multiple ciphertexts,
3. centralized memory management for all slots.

Memory for all slots is allocated once and then distributed among them.

Architectural notes

This structure represents the head of the pool and the entire slot system. The first field is a pointer to the array of slots (specifically, to its first element). During decapsulation, this array is scanned to find a free slot for processing a given ciphertext. This mechanism preserves parallelism while avoiding repeated allocations. The second field defines the number of slots and is specified by the user of the implementation during key initialization. All remaining fields are used exclusively by the pool destructor. They store pointers to allocated memory regions along with their sizes, allowing proper cleanup.

Design note

The pool owns all allocated memory and is fully responsible for its deallocation. Individual slots do not perform any independent memory allocations.

2.12.7. struct ml_kem_encaps_slot

struct ml_kem_encaps_slot {

```
    struct ml_kem_encaps_ctx *encaps_ctx;
    ml_kem_atomic_t is_free;
};
```

1. struct ml_kem_encaps_ctx *encaps_ctx - Pointer to the encapsulation structure of this variant
2. ml_kem_atomic_t is_free - variable for atomic access to this slot

Represents a single reusable encapsulation slot within the pool.

2.12.8. struct ml_kem_pool_encaps_ctx

```
// Pool of encapsulation slots
struct ml_kem_pool_encaps_ctx {
    struct ml_kem_encaps_slot *ml_kem_pool_512;  // pool for slots 512 level
    struct ml_kem_encaps_slot *ml_kem_pool_768;  // pool for slots 768 level
    struct ml_kem_encaps_slot *ml_kem_pool_1024; // pool for slots 1024 level
    size_t ml_kem_pool_count_512;              // number slots for 512 level
    size_t ml_kem_pool_count_768;              // number slots for 768 level
    size_t ml_kem_pool_count_1024;             // number slots for 1024 level

    // Allocated memory regions(it needs for destructor)
    u16 *all_u16_mem_512;
    u16 *all_u16_mem_768;
    u16 *all_u16_mem_1024;
    u8 *all_u8_mem_512;
    u8 *all_u8_mem_768;
    u8 *all_u8_mem_1024;
```

};

Represents the main encapsulation pool structure, managing reusable encapsulation slots and their associated memory resources for all three ML-KEM security levels.

The structure contains three independent arrays of encapsulation slots (ml_kem_pool_512, ml_kem_pool_768, and ml_kem_pool_1024), along with corresponding counters specifying the number of allocated slots for each security level.

Additionally, it maintains pointers to the contiguous memory regions allocated for each pool. The all_u16_mem_* fields reference the 16-bit polynomial working buffers, while the all_u8_mem_* fields reference the byte-oriented buffers used for serialized public keys, ciphertexts, and temporary XOF output.

These memory pointers are preserved to ensure proper memory management, allowing the pool destructor to securely wipe and release all allocated resources.

2.13. Header ml_kem_core_header.h

2.13.1. List of constant

The ML-KEM implementation uses a set of fixed parameters defined by the FIPS 203 standard, as well as auxiliary constants required for efficient computation and memory organization. The following constants (or groups of constants) are used:

1. ML_KEM_Q = 3329 — modulus of the *ring Z_q*.
2. ML_KEM_N = 256 — number of polynomial coefficients.

3. ML_KEM_COEFF_NORMALIZATION = 3303 — this constant is used after certain arithmetic operations on polynomials to normalize coefficients into *a valid representation in Z_q*. It effectively acts as a multiplicative factor in a normalization procedure, allowing values to be reduced into the valid range without using expensive division operations.
4. ML_KEM_SEED_BYTES = 32 — size of cryptographic seeds.
5. ML_KEM_MSG_COEFF = 1665 — used for bit-to-polynomial encoding of message bits (corresponds to ⌈q/2⌉).
6. ML_KEM_Q_RECIP_48 = (u64)0x13AFB7680BULL — precomputed reciprocal coefficient used for fast division; applied in the ML-KEM compression formula.
7. ML_KEM_Q_HALF = ML_KEM_Q / 2 — used for rounding and compression.
8. ML_KEM_MATRIX_XOF_BYTES = 576 — number of raw bytes generated per polynomial *for matrix A*.
9. ML_KEM_VECTOR_XOF_BYTES_L2 = 192 — number of raw bytes generated per polynomial for vectors when $\eta = 3$.
10. ML_KEM_VECTOR_XOF_BYTES_L34 = 128 — number of raw bytes generated per polynomial for vectors when $\eta = 2$.
11. ML_KEM_CBD_ETA_3 = 3 — value of η used in t*he CBD algorithm (for ML-KEM-512 during key generation).*
12. ML_KEM_CBD_ETA_2 = 2 — value of η used in *the CBD algorithm (for ML-KEM-768 and ML-KEM-1024 during key generation).*
13. RES_PUBL_PART_LVL_512 = 800, RES_PUBL_PART_LVL_768 = 1184, RES_PUBL_PART_LVL_1024 = 1568 — constants defining the length of the public key (serialized form) depending on the security level.

14. LEN_CIPHERTEXT_512 = 768, LEN_CIPHERTEXT_768 = 1088, LEN_CIPHERTEXT_1024 = 1568 — constants defining ciphertext length depending on the security level.

15. Internal error codes. This is done for the convenience of porting to different platforms, since there are environments where quite specific error handling and NULL and a simple negative code are not enough. Therefore, in API for return error using wrappers-functions for errors. These wrappers should handle internal error codes. If the error has an analogue code in POSIX, then it is assigned the corresponding code, if the error does not have this analogue and is specific to the implementation, then it is assigned a code outside the POSIX system.

    - ML_KEM_EINVAL 22 - incorrect data received
    - ML_KEM_ENOMEM 12 - out of memory
    - ML_KEM_EBUSY 16 - all slots are taken
    - ML_KEM_EAGAIN 11 - Resource temporarily unavailable(worth trying again)
    - ML_KEM_SYSTEM_ENTROPY_FAILED 134 - Error in the internal entropy function in the code of this implementation ML-KEM
    - ML_KEM_CALLBACK_ENTROPY_FAILED 135 - Entropy function transfer error
    - ML_KEM_ENCAPS_PARAM_ERR 136 - Error in encapsulation function (Since encapsulation returns a pointer, it was introduced to make

encapsulation errors more informative for environments that may not be comfortable with simply returning NULL)

Security Level Enumeration

The implementation also defines the following enumeration:

```
enum ml_kem_k {
   ML_KEM_512 = 2,
   ML_KEM_768 = 3,
   ML_KEM_1024 = 4,
};
```

This enumeration serves two purposes:

1. Defines the security level of the algorithm.
2. Defines the parameter k, which determines the dimensionality of vectors and matrices used in computations and directly depends on the selected security level.

The enum ml_kem_k type is used not only to select the security level, but also to:

- determine algorithm parameters,
- select appropriate constants,
- and control computation paths depending on the configuration.

2.13.2. Inline functions in header

1. u16 ml_kem_barrett_reduce(u32 a) - Barrett reduction: reduces a 32-bit value modulo q in constant time

2. u16 ml_kem_multipl_mod(u16 a, u16 b) - Modular multiplication: (a * b) mod q

3. u16 ml_kem_add_mod(u16 a, u16 b) - Modular addition: (a + b) mod q

4. u16 ml_kem_sub_mod(u16 a, u16 b) - Modular subtraction: (a - b) mod q

5. u32 load24_littleendian(const u8 *x) and void cbd3(u16 *r, const u8 *buf) - Functions for CBD algorithm, eta == 3

6. load32_littleendian(const u8 *x) and void cbd2(u16 *r, const u8 *buf) - Functions for CBD algorithm, eta == 2

7. void ml_kem_gen_polynomial_for_cbd(u16 *poly, const u8 *stream, const u8 eta) - The main function of the wrapper for CBD

8. int ml_kem_gen_polynomial_for_matrix(u16 *poly, const u8 *stream, int suma_bytes) - Functions for get coefficients for matrix A(see algorithm 7 in standard)

2.13.3. Preprocessor functions and directives for porting to different platforms

The #elif directive specifies a specific flag to be passed during the build and compilation process, indicating the environment under which this implementation will run. The #elif directive selects the header required for the specified platform based on the received flag. Therefore, this implementation, in its pure form, is not intended to run on any platform; a port file must be written for it. This was done because it is impossible to port absolutely everything to C11, there are certain elements that are platform-specific.

Also, according to the passed flag and the selected port header, the following platform-dependent functions must be described in the header and code of the port itself. Only wrappers are defined here, they must be described within the specified prototypes in the port files, which should be compiled together with all implementation files. The prototypes of the platform-dependent functions are as follows:

1. typedef int (*ml_kem_entropy_fn)(void *buf, size_t len) - prototype for entropy function
2. int ml_kem_entropy(void *buf, size_t len) - entropy function
3. void *ml_kem_alloc(size_t size_alloc) - allocation memory function
4. void ml_kem_free(void *ptr) - free memory function
5. void ml_kem_memzero(void *ptr, size_t len) - secure zeroing function for secrets
6. bool ml_kem_try_acquire_slot(ml_kem_atomic_t *slot) - for atomic receiving slot
7. void ml_kem_release_slot(ml_kem_atomic_t *slot) - for atomic free slot
8. void ml_kem_atomic_init_slot(ml_kem_atomic_t *slot) - create atomic variable for slot
9. ml_kem_pool_decaps_ctx *err_create_keys(const int err) - error handling in the key generation API function
10. u8 *err_encaps(const int err) - error handling in the encapsulation API function
11. int err_decaps(const int err) - error handling in the decapsulation API function
12. int err_get_pk(const int err) - error handling in the API function for obtaining a public key

2.14. High-Level Execution Flow of ML-KEM in This Architecture

The operation of ML-KEM in this implementation is logically divided into two execution flows:

1. Encapsulation flow — the party that:
    - generates a message,
    - encapsulates the message,
    - sends the ciphertext.

2. Decapsulation flow — the party that:
    - generates keys,
    - receives a message in the form of ciphertext,
    - decapsulates the message.

2.14.1. Encapsulation Flow

The encapsulation flow operates as follows for each generated message:

1. Memory is allocated for the encapsulation structure (ml_kem_encaps_ctx) using a function from the ml_kem_encaps.c module.
2. The main encapsulation function from ml_kem_encaps.c is executed and produces two results:
    - the shared secret,
    - the ciphertext.
3. The encapsulation flow:
    - stores the shared secret internally (from the corresponding field of the structure),
    - and uses the ciphertext (also stored in the structure) as the message to be sent.
4. Once all required data is obtained and processed, the destructor of the allocated structure is called.

Encapsulation model

The encapsulation flow is stateless and one-shot:

- a new context is created for each message,
- no memory reuse is performed,

- the context is destroyed immediately after use.

This ensures:

- full isolation between operations,
- no dependency on previous executions,
- no need to preserve state between calls.

2.14.2. Encapsulation Flow with pool

The pool-based encapsulation flow operates in the same manner as the standard single-instance encapsulation described in the previous section. The main difference lies in memory management: instead of allocating and destroying an encapsulation context for each operation, the implementation maintains a pool of reusable encapsulation slots, with all required working memory pre-allocated.

To initiate an encapsulation operation for a given public key, an available slot is acquired atomically from the pool corresponding to the required ML-KEM security level. The standard encapsulation procedure is then performed using the acquired slot's context. Once the ciphertext and shared secret have been copied to the caller-provided buffers, sensitive temporary data is wiped, and the slot is atomically released for subsequent operations. If no slot is available, the function returns -EBUSY without waiting.

This mechanism is intended for scenarios where the encapsulating entity handles a high volume of encapsulation requests, potentially involving numerous distinct public keys. By reusing pre-allocated contexts, the implementation avoids repeated memory allocations and supports concurrent encapsulation operations.

The pool is initialized using create_encaps_pool(), which allows the number of slots

to be configured independently for each ML-KEM security level. Once the pool is no longer required and all active operations have completed, its resources are released using ml_kem_encaps_pool_destroy().

2.14.3. Decapsulation Flow

The decapsulation flow operates on a stream of ciphertexts associated with a specific key. Before describing the functions, it is recommended to familiarize yourself with diagrams 4.13., 4.14., 4.15.a, 4.15.b, 4.16. The full key lifecycle is as follows:

Key generation phase

1. Memory is allocated for key generation structures:
    - ml_kem_temp (temporary),
    - ml_kem_ctx (persistent), using a function from the ml_kem_create_keys.c module.
2. The main key generation function produces the result in the persistent key structure (ml_kem_ctx). This structure is never modified afterward and is only zeroized and destroyed when the keys are no longer needed.
3. The temporary key generation structure (ml_kem_temp) must be immediately zeroized and destroyed after use, as it contains sensitive data.

Pool initialization phase

4. Using the ml_kem_pool.c module, a pool of slots is allocated (ml_kem_pool_decaps_ctx). Each slot (ml_kem_decaps_ctx) contains:
    - buffers,
    - a decryption context (ml_kem_decrypt_ctx),
    - an encapsulation context (ml_kem_encaps_ctx).

Their sizes and types are aligned with:

- ML-KEM parameters,
- and the structure of persistent key data.

The pool size (number of slots) is specified by the user during key initialization.

5. Each slot contains a pointer to the same persistent key structure.

   This pointer is the only field that is never cleared.

Decapsulation execution phase

6. During decapsulation:

   - no memory allocation is performed,
   - instead, the preallocated pool is used.

   Incoming ciphertexts are processed as follows:

   Each ciphertext may trigger decapsulation independently and in parallel for the same key:

   6.1. A free slot is searched.

   6.2. If no free slot is available, the function returns -EBUSY.
   If a slot is available, it is atomically acquired.

   6.3. The decapsulation process is executed:

   - Decryption (via ml_kem_decrypt_ctx)
   - Re-encapsulation (via ml_kem_encaps_ctx)
   - Ciphertext comparison and result derivation

   6.4. After completion:

   - the persistent key structure remains unchanged,
   - all slot-related data is zeroized except the pointer to the key structure.

6.5. The slot is atomically released and becomes available for reuse.

Destruction phase

7. When the keys are no longer needed:
    - the pool is destroyed (fully zeroized and deallocated),
    - then the persistent key structure is also zeroized and destroyed.

Core architectural idea

Unlike encapsulation, which is a one-shot operation with dynamic allocation, decapsulation is implemented as a *stream-processing system* using a preallocated pool with full memory reuse.

Data model

The architecture is fundamentally based on separating data into two categories:

1. Persistent data (key-related)
2. Per-ciphertext data

Per-ciphertext data:

- must match the size and structure defined by the selected security level,
- and is organized as reusable slot memory.

Each slot:

- contains all data required to process a single ciphertext,
- except for the pointer to the persistent key structure.

The key structure:

- is shared across all slots,
- is read-only during decapsulation,
- is never modified after creation.

Resulting properties

This design allows:

- zeroizing slot data after each operation,
- reusing memory safely across multiple ciphertexts,
- maintaining strict separation between persistent and ephemeral data,
- enabling parallel processing without additional allocations.

2.14.4. Memory Reuse Strategy

Memory reuse is implemented in both decapsulation and encapsulation flows through preallocated pools of reusable contexts. The main purpose of this approach is to eliminate repeated memory allocations during cryptographic operations and support parallel execution.

Decapsulation always uses a preallocated pool associated with a specific key pair. Each slot contains the working memory required for decryption and re-encapsulation, while all slots share the same persistent key context. This approach is particularly useful for systems that process a large number of incoming ciphertexts, where:

- frequent memory allocations introduce unnecessary overhead,
- parallel processing is important for handling concurrent requests,
- reusing allocated memory reduces runtime memory management costs.

Encapsulation supports two execution modes:

1. **Standalone mode:** A temporary encapsulation context is allocated for each operation and destroyed after completion. This mode is suitable for occasional encapsulation operations where maintaining a reusable pool is unnecessary.
2. **Pool mode:** Encapsulation contexts and their working buffers are allocated in advance and reused across operations. Separate pools can be configured for

ML-KEM-512, ML-KEM-768, and ML-KEM-1024, allowing concurrent encapsulation operations with different public keys without repeated allocations.

In both pool-based modes, an available slot is acquired atomically before the cryptographic operation. After completion, sensitive temporary data is zeroized, and the slot is released for subsequent use. If all slots are occupied, the operation returns an error instead of waiting for a free slot.

The primary trade-off of memory reuse is increased memory consumption during initialization. However, this allows the implementation to avoid dynamic allocations during encapsulation and decapsulation operations when using the corresponding pools, while maintaining support for concurrent execution.

2.14.5. Design Rationale

The architectural approach used in this implementation is based on addressing a key characteristic of ML-KEM: decapsulation operates on relatively large data structures and is often executed repeatedly. Allocating and freeing memory for each decapsulation operation would introduce significant overhead, especially in constrained or performance-sensitive environments such as:

- operating system kernels,
- RTOS-based systems,
- embedded platforms.

To mitigate this, the implementation introduces a preallocated pool of slots, where each slot contains all required data for processing a single ciphertext. This design provides the following advantages:

- eliminates per-operation memory allocation,
- enables parallel processing of multiple ciphertexts,
- provides predictable and stable performance characteristics,
- centralizes memory management.

Each slot is capable of processing any ciphertext associated with a given key, and the pool size can be configured during key generation. This makes the system flexible and adaptable to different workloads.

Trade-offs

The primary trade-off of this design is increased memory usage. All required memory is allocated upfront during key initialization, which may result in a larger overall memory footprint compared to per-operation allocation models.However: this allocation happens only once, and even memory-constrained environments are expected to accommodate a moderate amount of memory (e.g., tens of kilobytes) in exchange for improved performance and maintained parallelism.

Portability considerations

This implementation is written in pure C and is designed to stay as close as possible to the C standard and the ML-KEM specification. Achieving full universality across all environments is not feasible due to differences in:

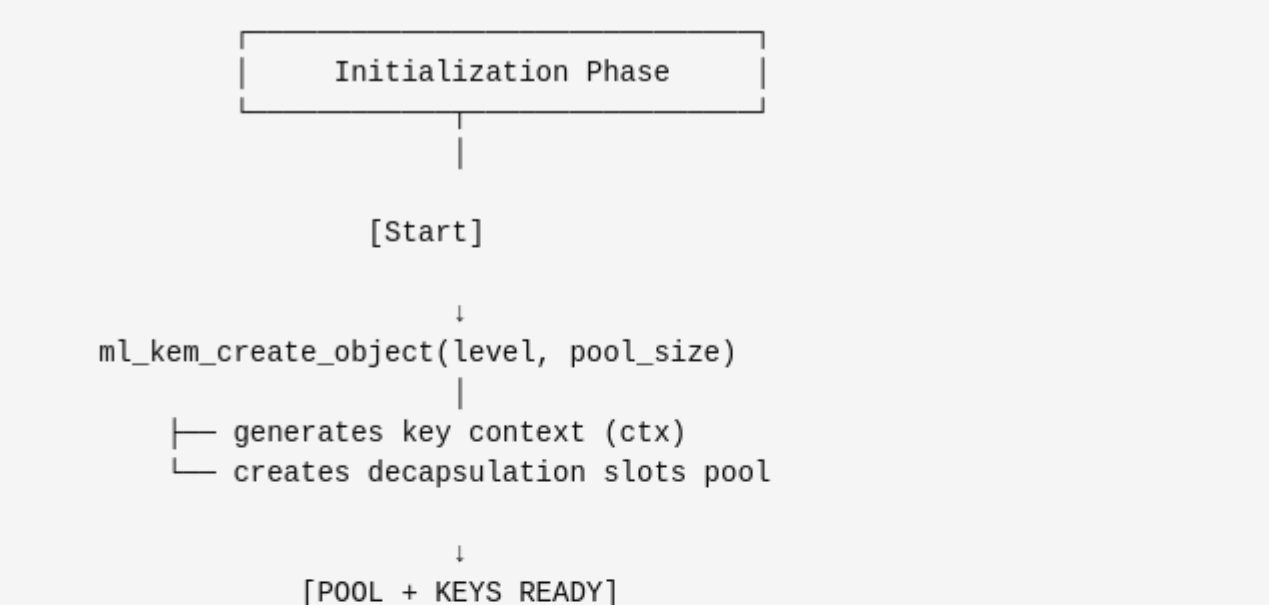
- memory allocation mechanisms,
- available standard libraries,
- entropy sources.

However, the design aims to:

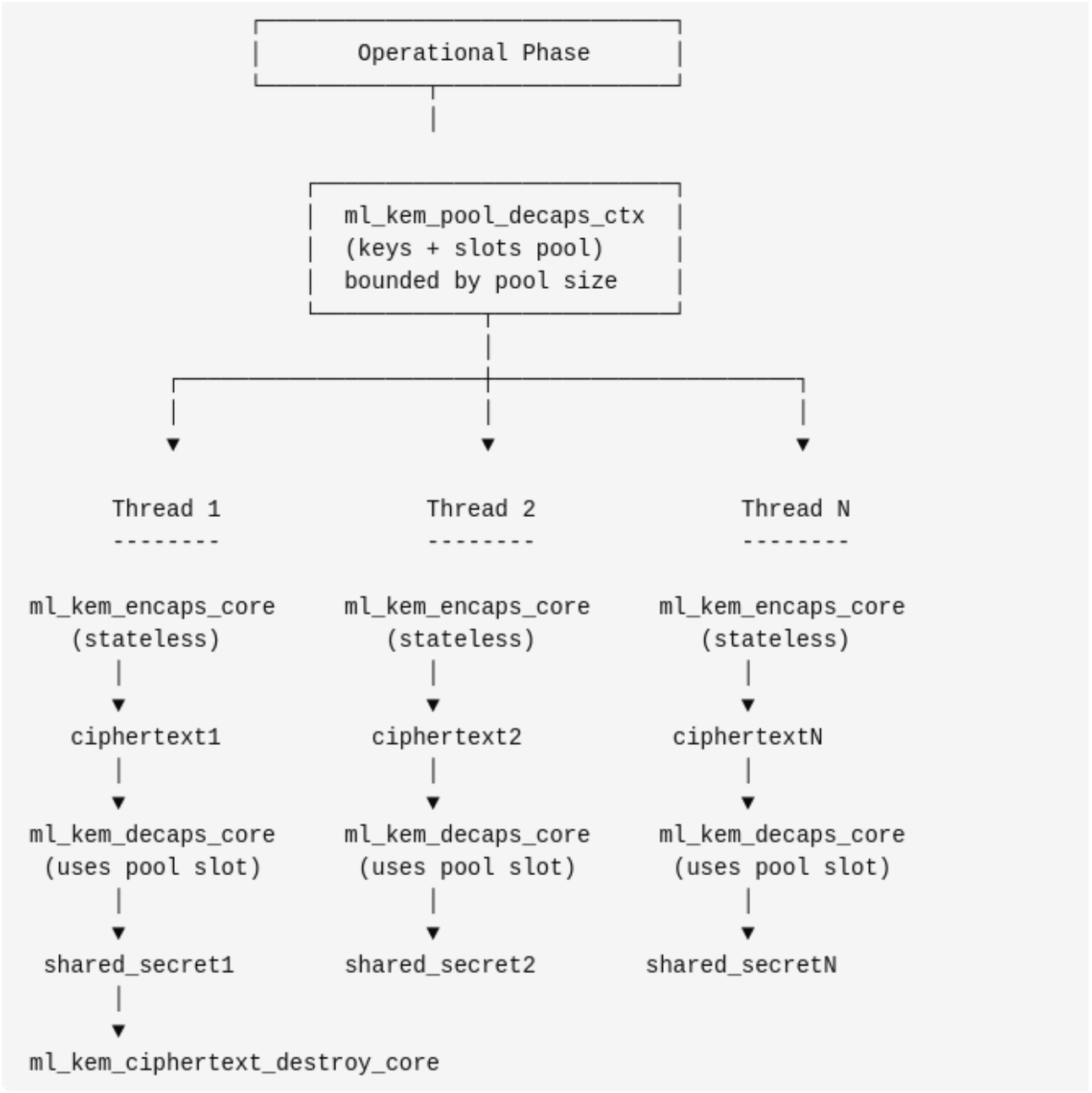
- simplify porting to different environments,
- minimize dependencies,
- and allow integration of hardware acceleration (e.g., via preprocessor directives) where available.

3. IMPLEMENTATION ARCHITECTURE DIAGRAMS

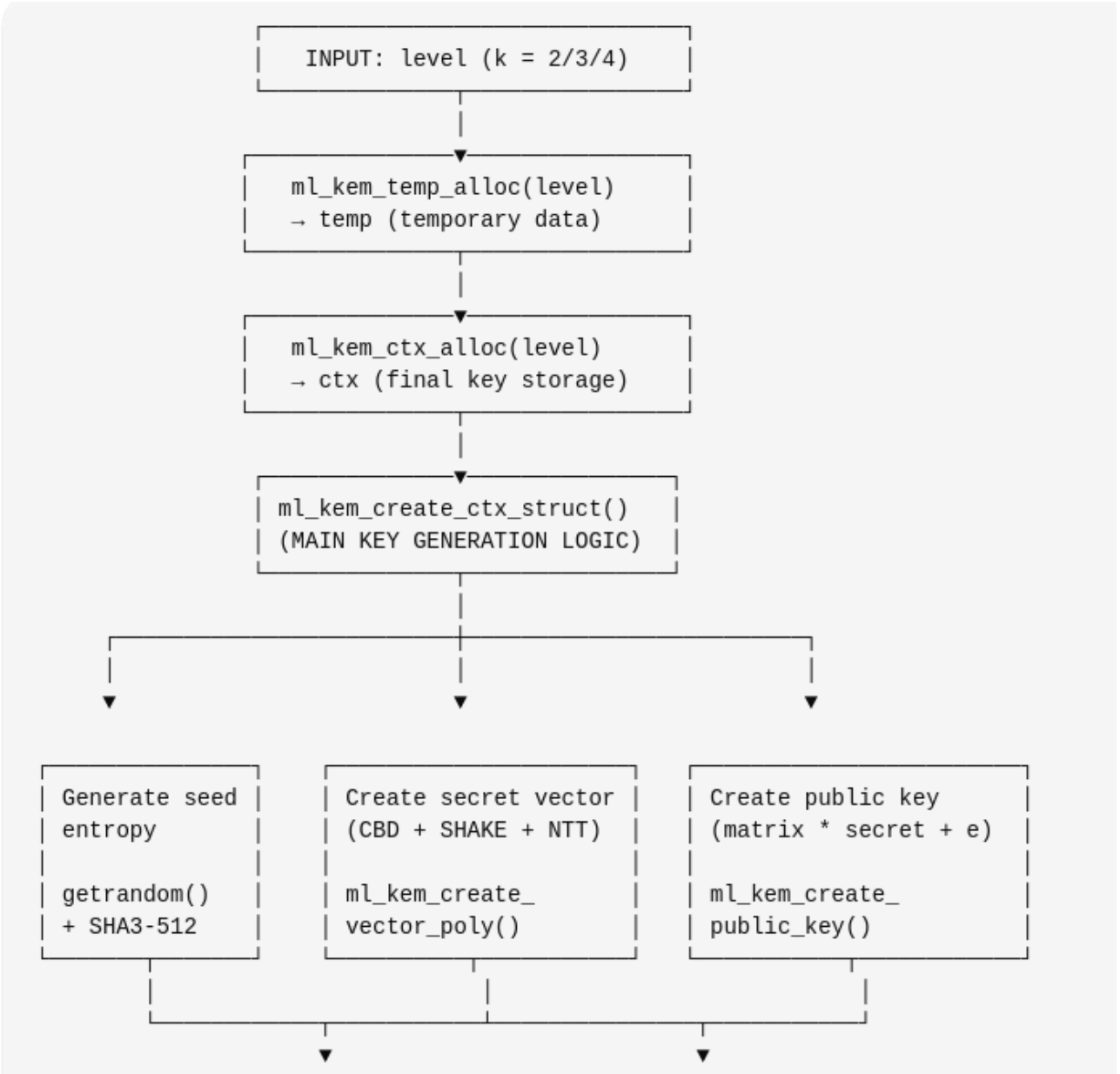
This section provides visual diagrams illustrating the architecture of the ML-KEM implementation and the sequences of execution of the main operations described in Section 3.



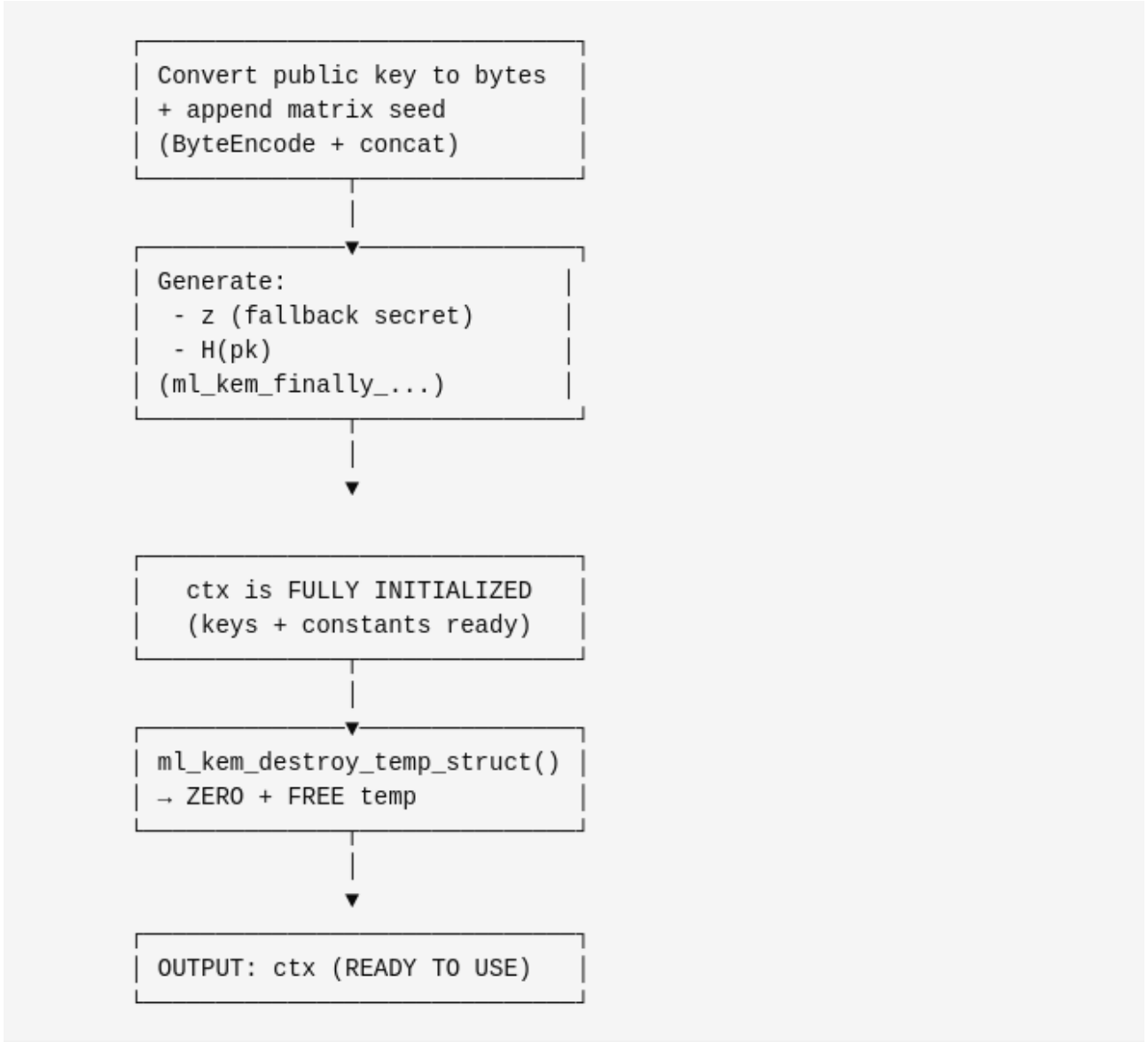
3.1.Object Lifecycle



3.2. Parallel Runtime Flow



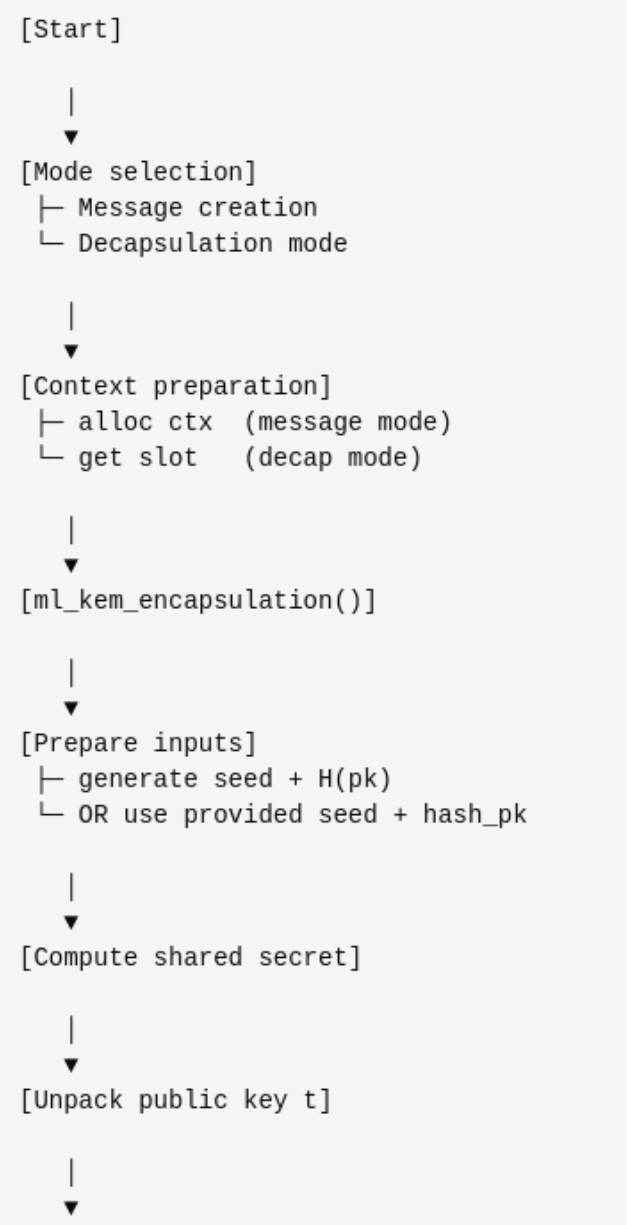
3.3a — General key generation scheme



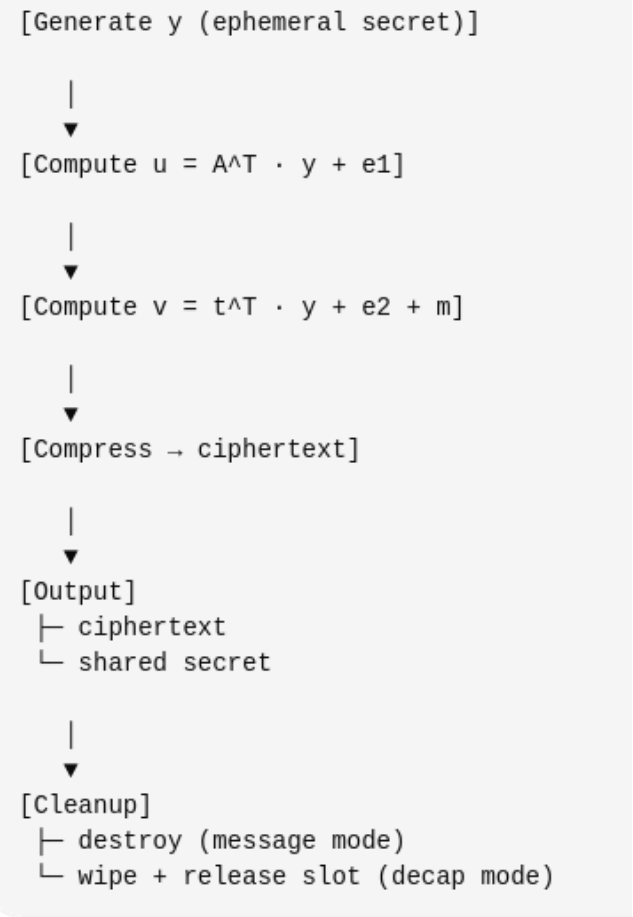
3.3b General key generation scheme(continued)

```
ml_kem_create_ctx_struct(temp, ctx)
|
├── [1] Validate input
|
├── [2] Generate entropy
|     └── ml_kem_generation_entropy()
|         ├── getrandom(32 bytes)
|         ├── append k
|         └── SHA3-512 → (seed_rho, seed_sigma)
|
├── [3] Generate secret vector s
|     └── ml_kem_create_vector_poly()
|         ├── SHAKE256(seed_sigma || counter)
|         ├── ml_kem_gen_polynomial_for_cbd()
|         └── NTT transform
|
├── [4] Generate public key t
|     └── ml_kem_create_public_key()
|         ├── Generate A using seed_rho
|         |   └── ml_kem_gen_polynomial_for_matrix()
|         |
|         ├── Multiply A × s
|         ├── Add noise e
|         └── Result → t
|
├── [5] Serialize public key
|     └── ml_kem_convert_to_bytes_format_and_copy()
|         ├── pack coefficients
|         └── append seed_rho
|
├── [6] Finalization
|     └── ml_kem_finally_create_z_and_h_pk()
|         ├── generate z
|         ├── compute H(pk)
|         └── store in ctx
|
└── [7] Done
```

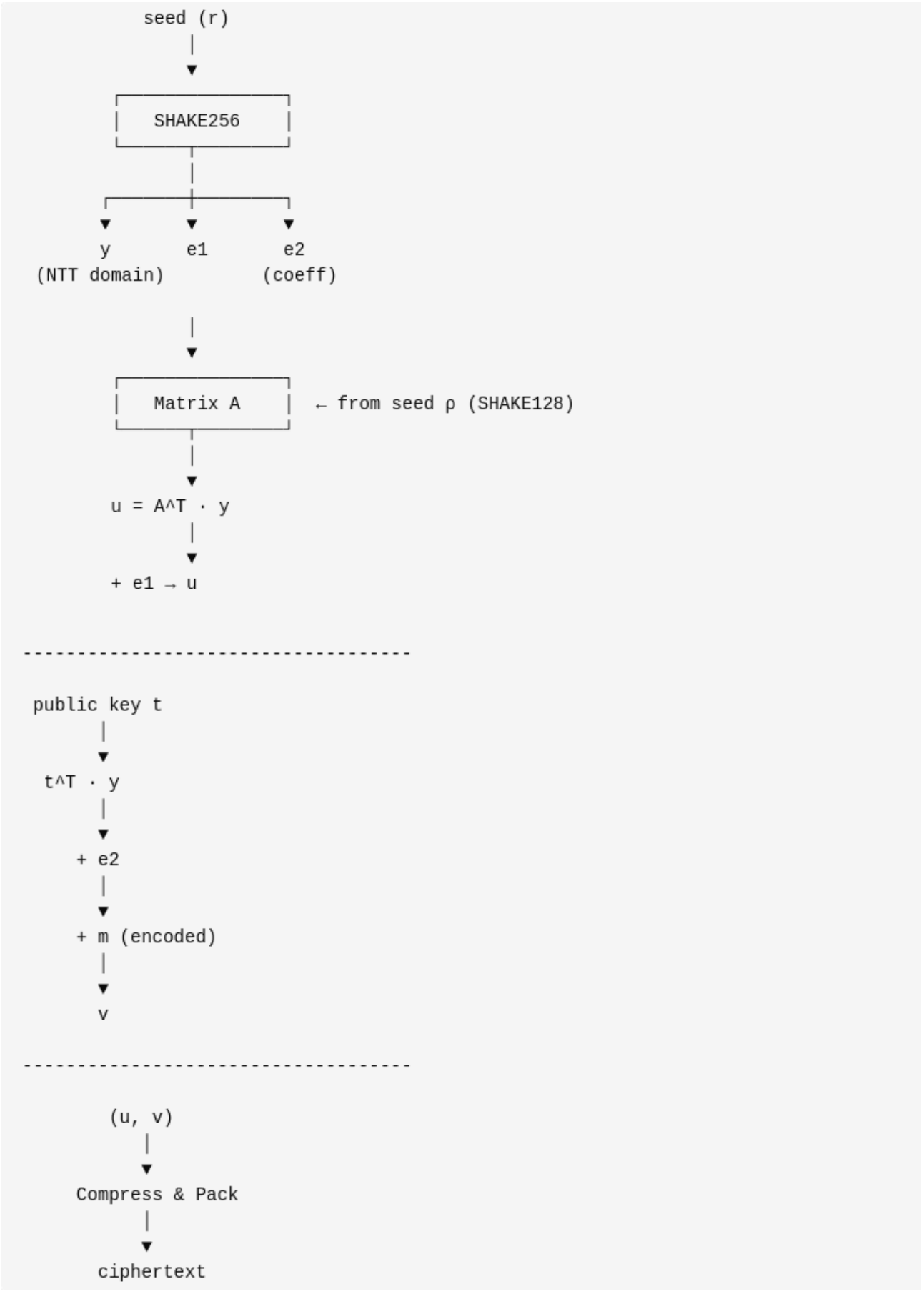
3.4 Key generation, detailed diagram of the module functions



3.5a. Encapsulation execution sequence



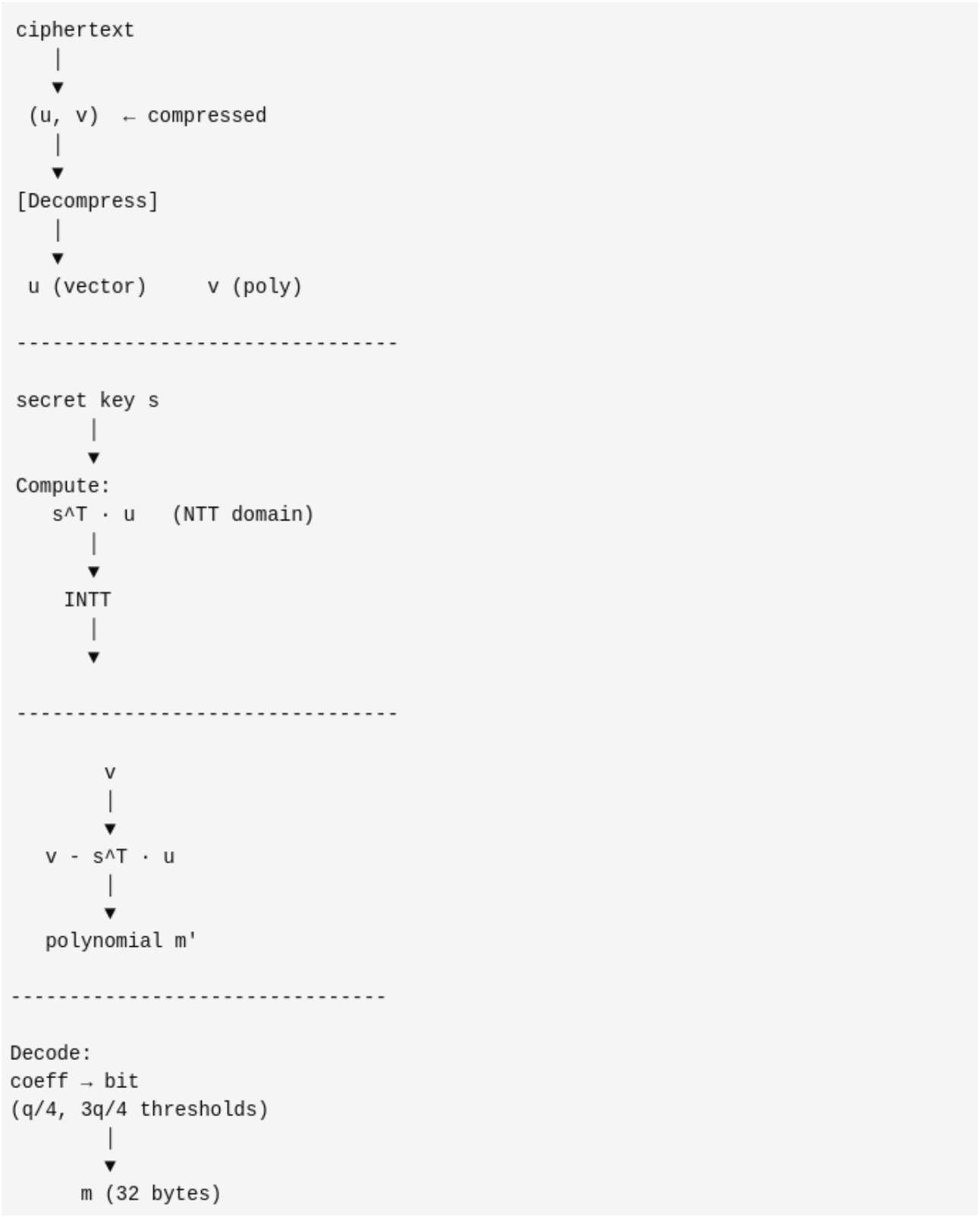
3.5b. Encapsulation Execution Sequence (continued)



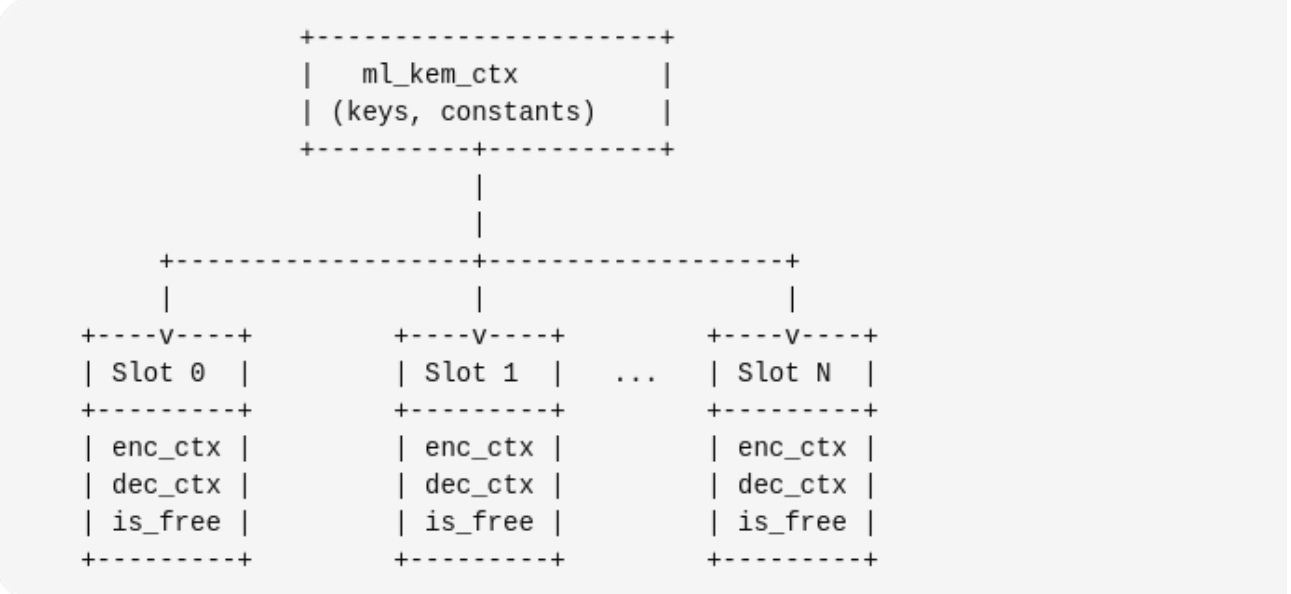
3.6 The internal mathematics of encapsulation



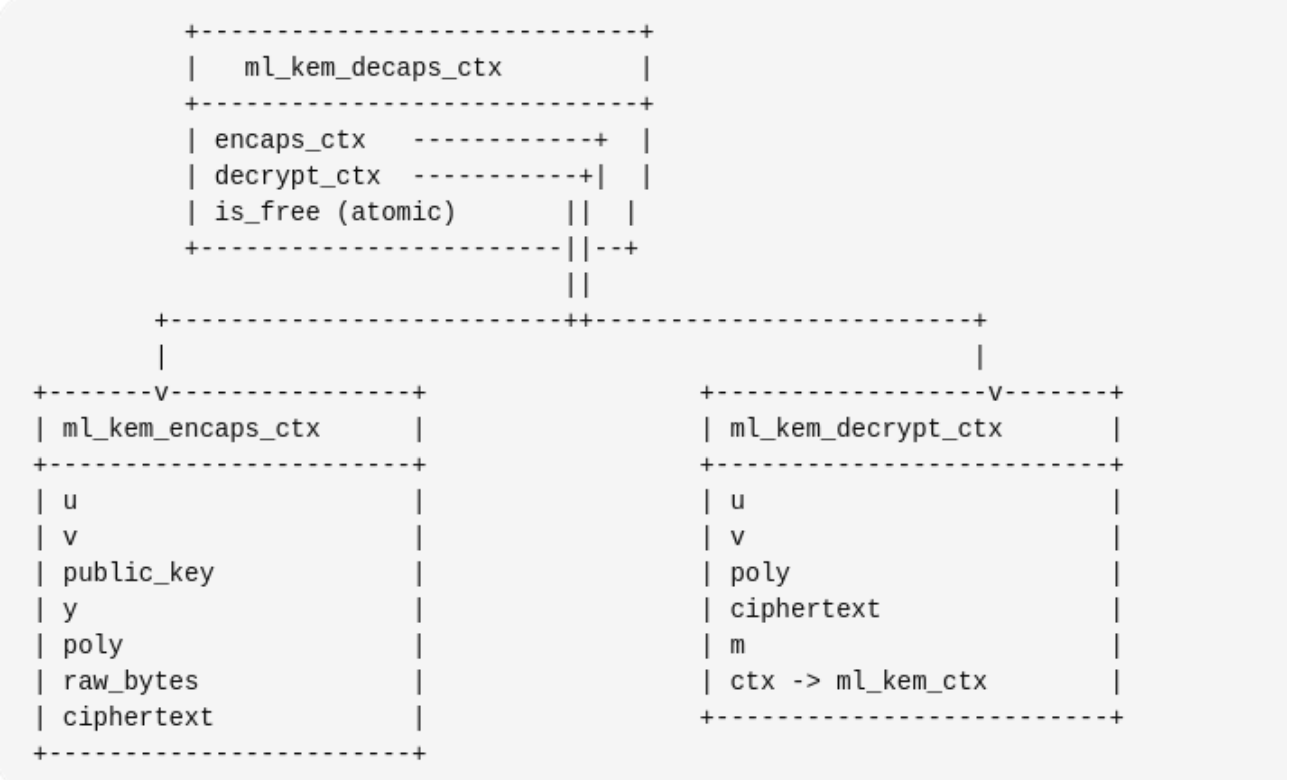
3.7 Decryption execution sequence



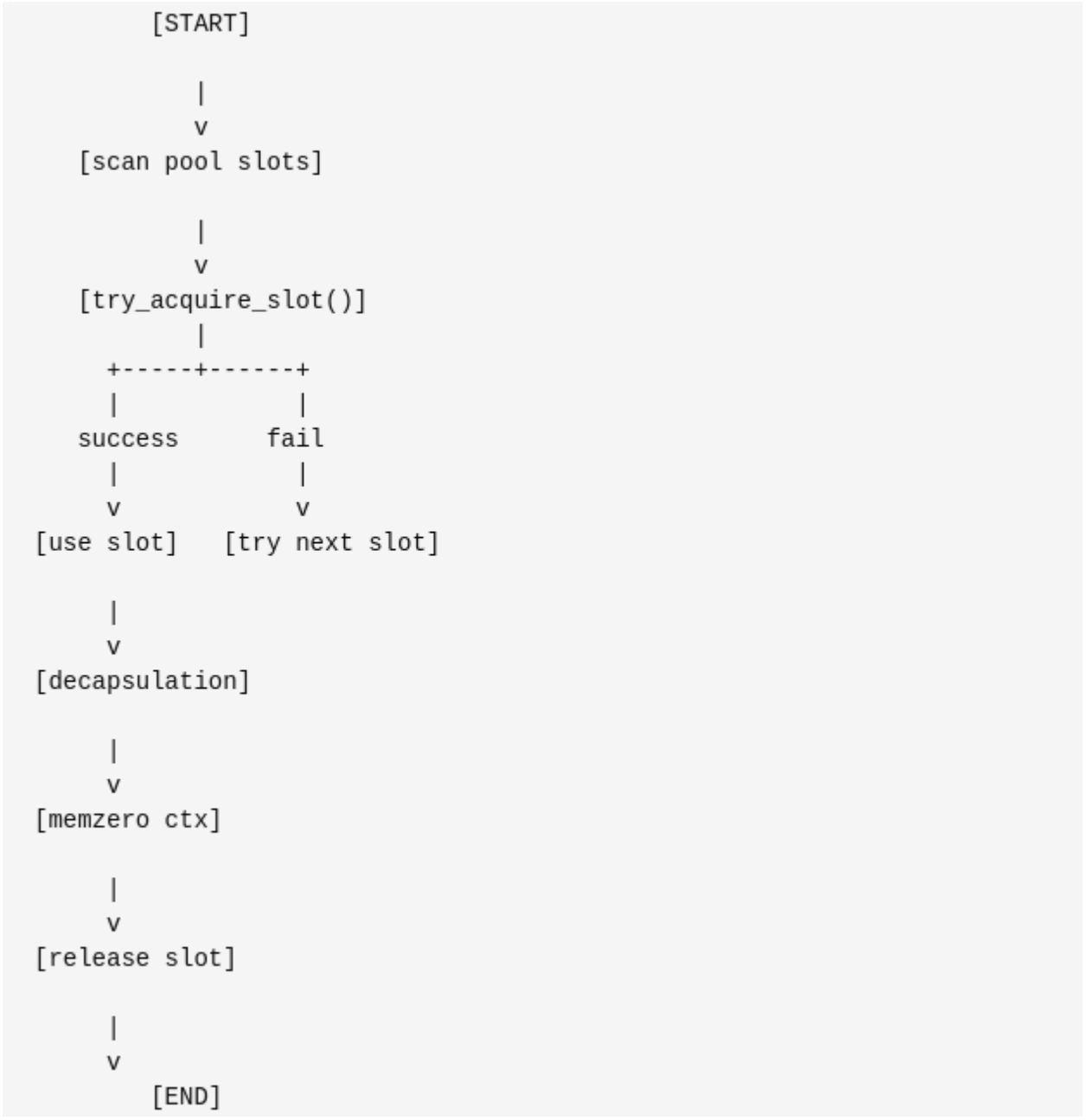
3.8 The internal mathematics of decryption



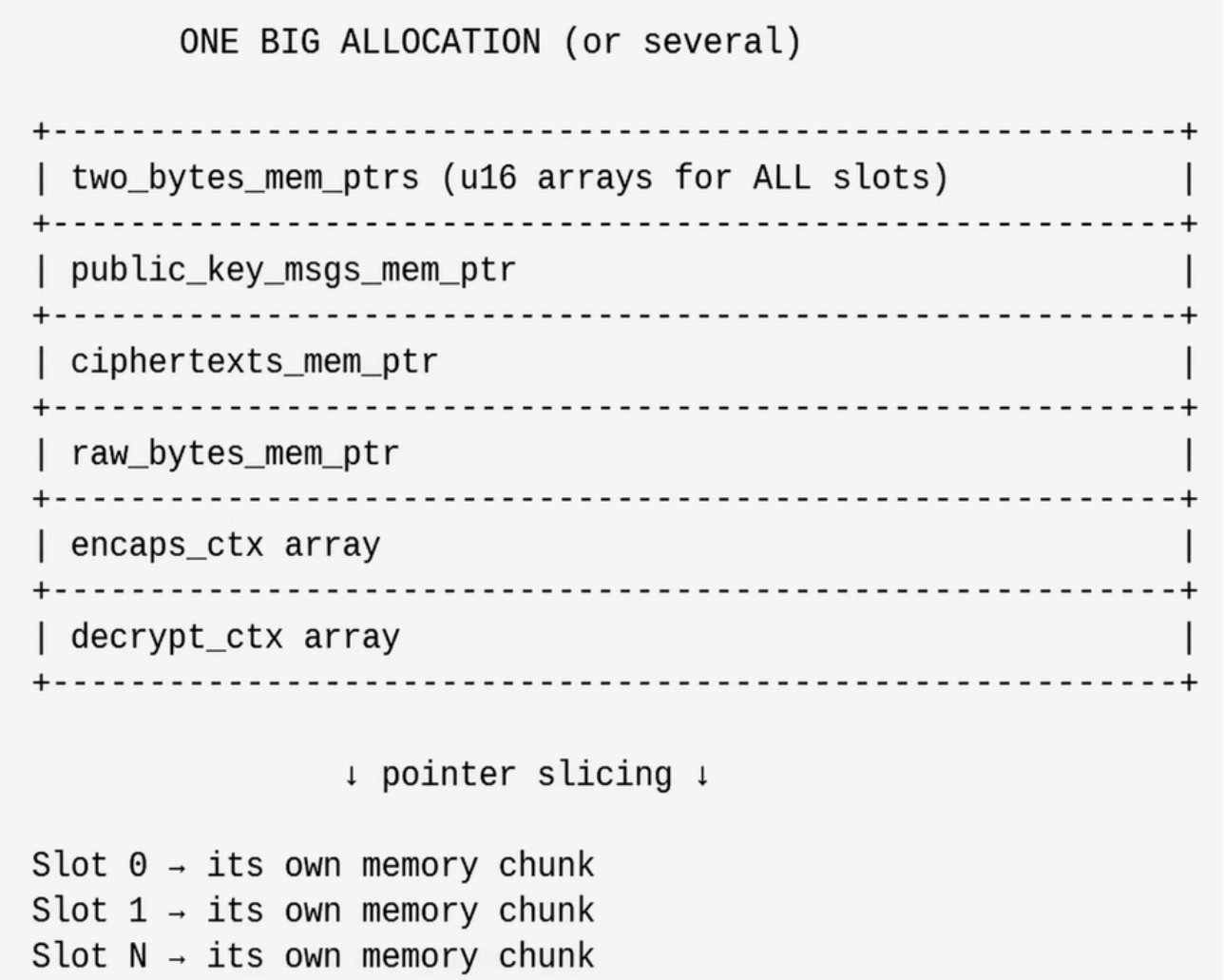
3.9. General pool architecture



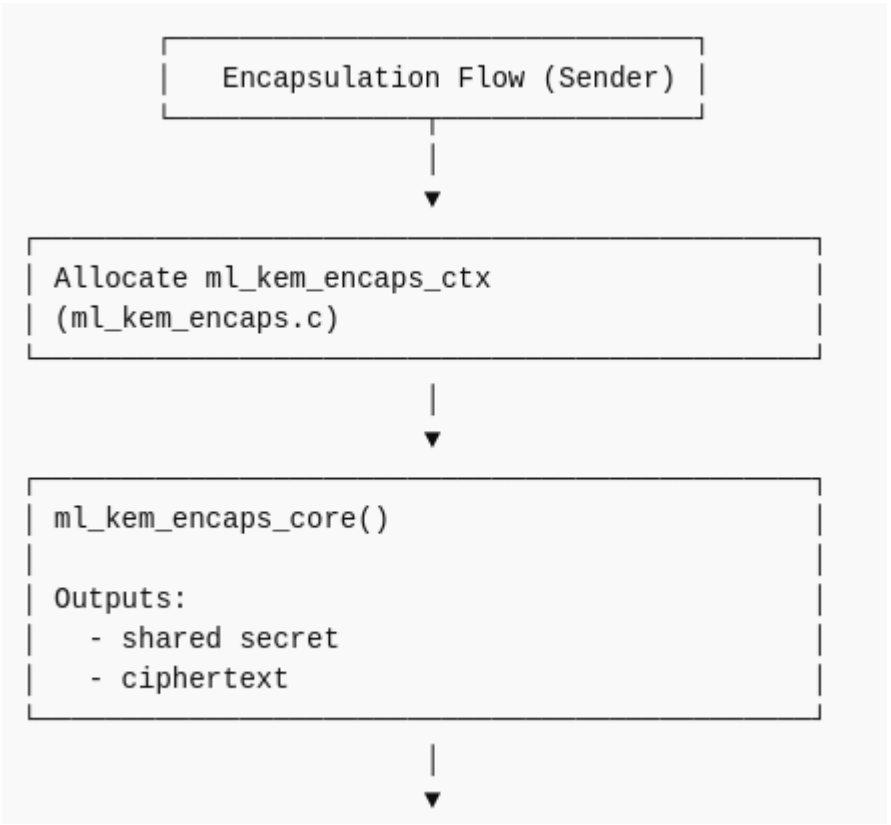
3.10. Internal structure of one slot

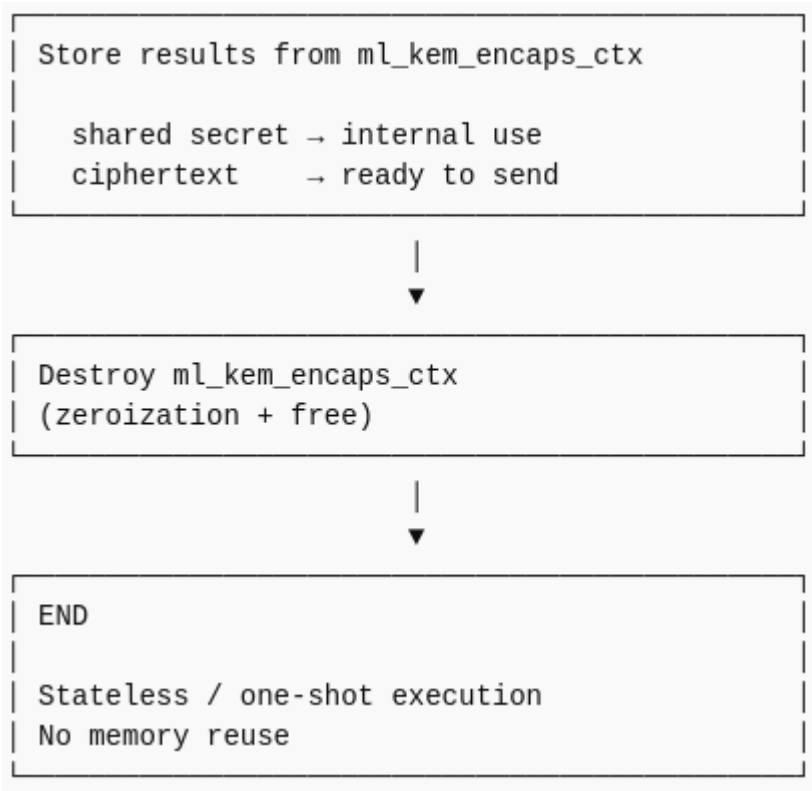


3.11. Slot Lifecycle

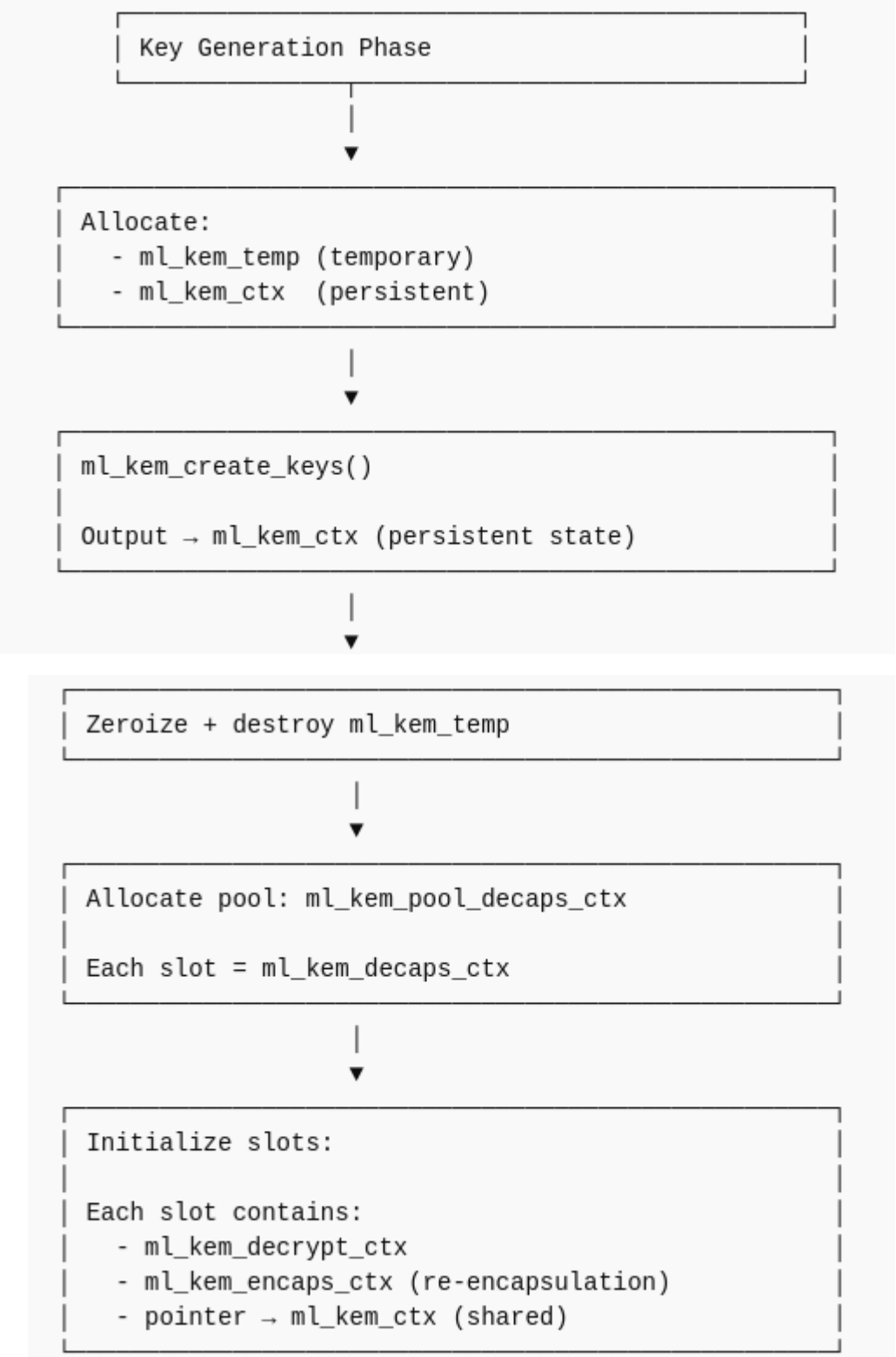


3.12. Pool and slot memory layout

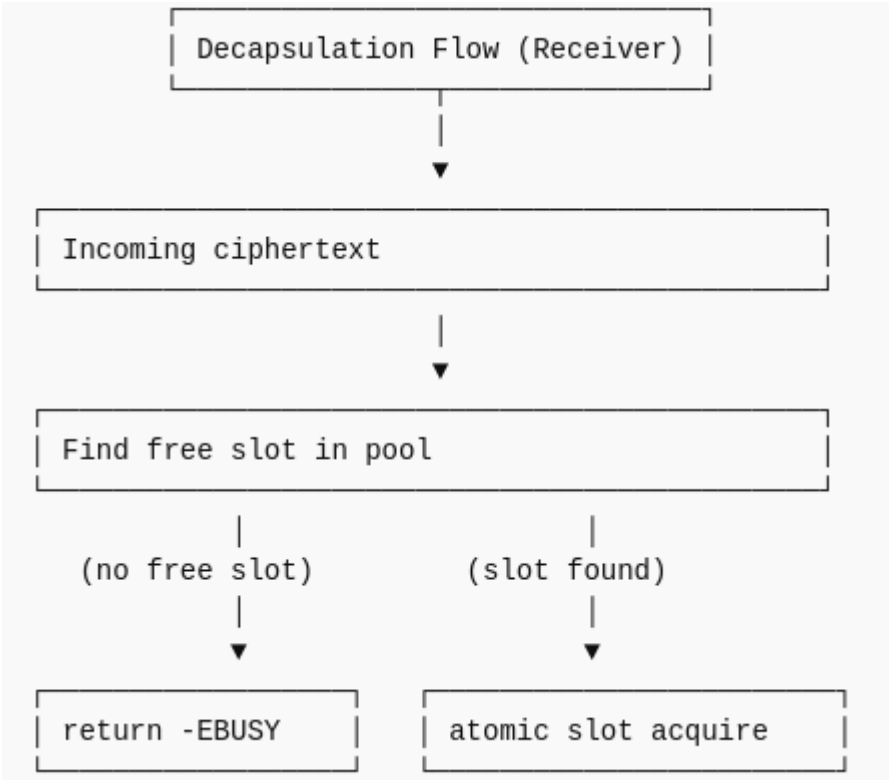

Encapsulation Flow (Sender)
Allocate ml_kem_encaps_ctx
(ml_kem_encaps.c)
ml_kem_encaps_core()
Outputs:
- shared secret
- ciphertext


Store results from ml_kem_encaps_ctx
shared secret → internal use
ciphertext → ready to send
Destroy ml_kem_encaps_ctx
(zeroization + free)
END
Stateless / one-shot execution
No memory reuse

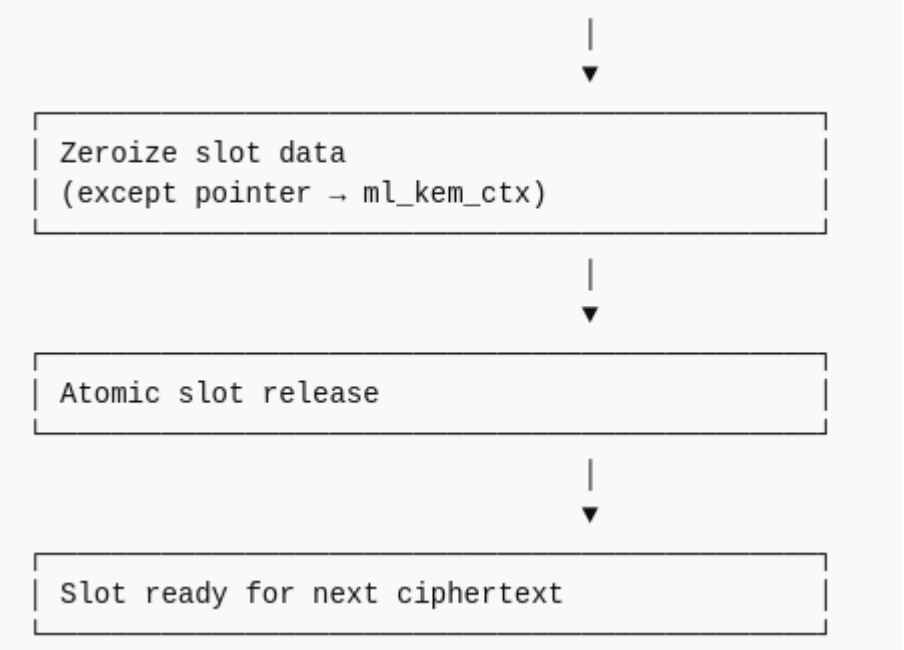
3.13. An encapsulation flow is an isolated, one-time execution flow that does not preserve state between calls and does not use a memory reuse mechanism.



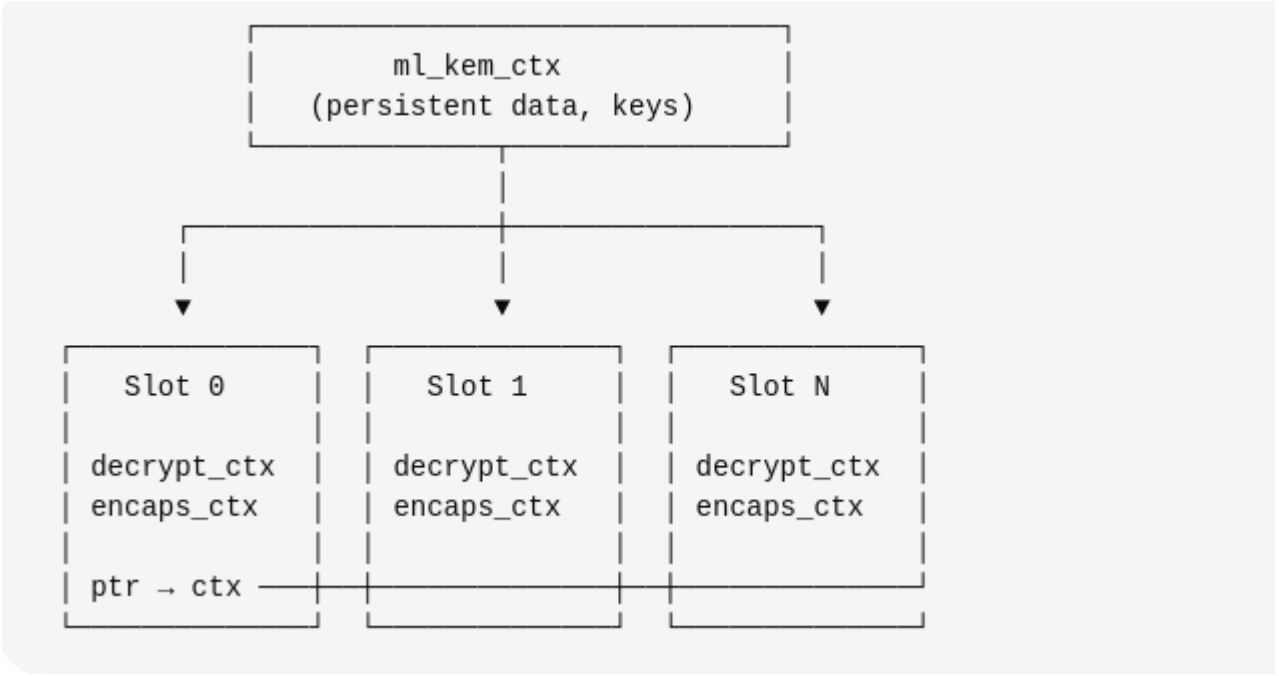
3.14. Decapsulation Flow. Initialization.



3.15.a. Runtime Decapsulation Flow



3.15.b. Runtime Decapsulation Flow(continue)



3.16. Memory Model